RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 Fundamentals of Data Science

Laboratory Record Note Book

Name :	Rithanya Selvakumar
Year / Branch / Section :	2nd year CSE 'G'
University Register No. :	2116250701597
College Roll No. :	250701597
Semester :	3
Academic Year :	2025-29

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 Fundamentals of Data Science

NAME	Rithanya Selvakumar
ROLL NO	250701597
YEAR	2nd year
SEC	G

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: Rithanya Selvakumar

Academic Year: 2025-29 Semester: 3 Branch: CSE

Register No. 2116250701597

Certified that this is the bonafide record of work done by the above student in theFundamentals of Data Science [CS23334] Laboratory during the academic year 2025 – 2026.

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……

Internal Examiner External Examiner

INDEX

SlNO	Experiment	Date
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	04/08/2026
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	04/08/2026
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	04/08/2026
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	04/08/2026
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	25/08/2026
3.a	Conduct an experiment to show data visualization using line plot	25/08/2026
3.b	Conduct an experiment to show data visualization using bar chart	25/08/2026
3.c	Conduct an experiment to show data visualization using pie chart	25/08/2026
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	25/08/2026

5	Data Collection and Initial Exploration	18/09/2026
6	Time Series Analysis of Website Traffic	18/09/2026
7	Random Sampling and Sampling Distribution	22/09/2026
8	Conduct Z test for the Data given using Python	22/09/2026
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	22/09/2026
10	Experiment to understand array function in Data science.	29/09/2026
11	Experiment to understand pandas library use cases in Data science.	29/09/2026
12	Experiment to detect outliers in a given data set.	29/09/2026
13	Experiment to handle missing values and inappropriate data in a given data set.	29/09/2026
14	Experiment to understand feature scaling	02/10/2026
15	Experiment to understand the data preprocessing in Data science	02/10/2026
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	02/10/2026
17	Experiment to understand Linear Regression for a given data set.	02/10/2026
18	Experiment to understand KNN algorithm for a given data set.	06/10/2026
19	Experiment to understand Logistic Regression for a given data set.	06/10/2026
20	Experiment to understand K-means clustering algorithm for a given data set	06/10/2026

Exp No:1.a Analyze the trend of data science job postings over the last

Decade

AIM:

Analyse the trend of data science job postings over the last

Decade

Description:

Use web scraping (e.g., Beautiful Soup) or APIs (e.g., LinkedIn API) to gather

data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

CODE:

```
import pandas as pd

import matplotlib.pyplot as plt

data = {'Year': list(range(2011, 2022)),

'Job Postings': [150, 300, 450, 600, 900, 1200, 1500, 2000, 2500, 3300, 4000]}


df = pd.DataFrame(data)

plt.plot(df['Year'], df['Job Postings'], marker='o')

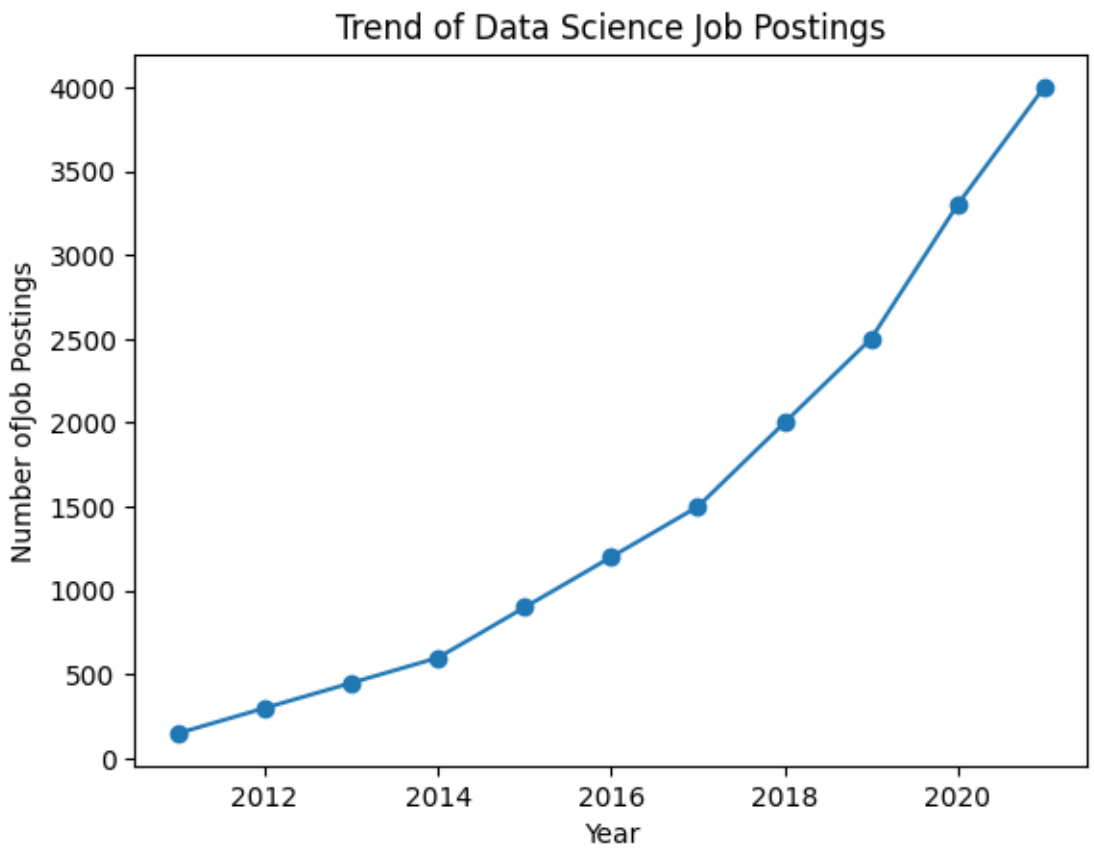
plt.title('Trend of Data Science Job Postings')

plt.xlabel('Year')

plt.ylabel('Number ofJob Postings')

plt.show()
```

OUTPUT:



RESULT: The graph shows that data science job postings increased from 2009 to 2021.The highest number of job postings was in 2021.

Exp No:1.b Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

AIM:

Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from

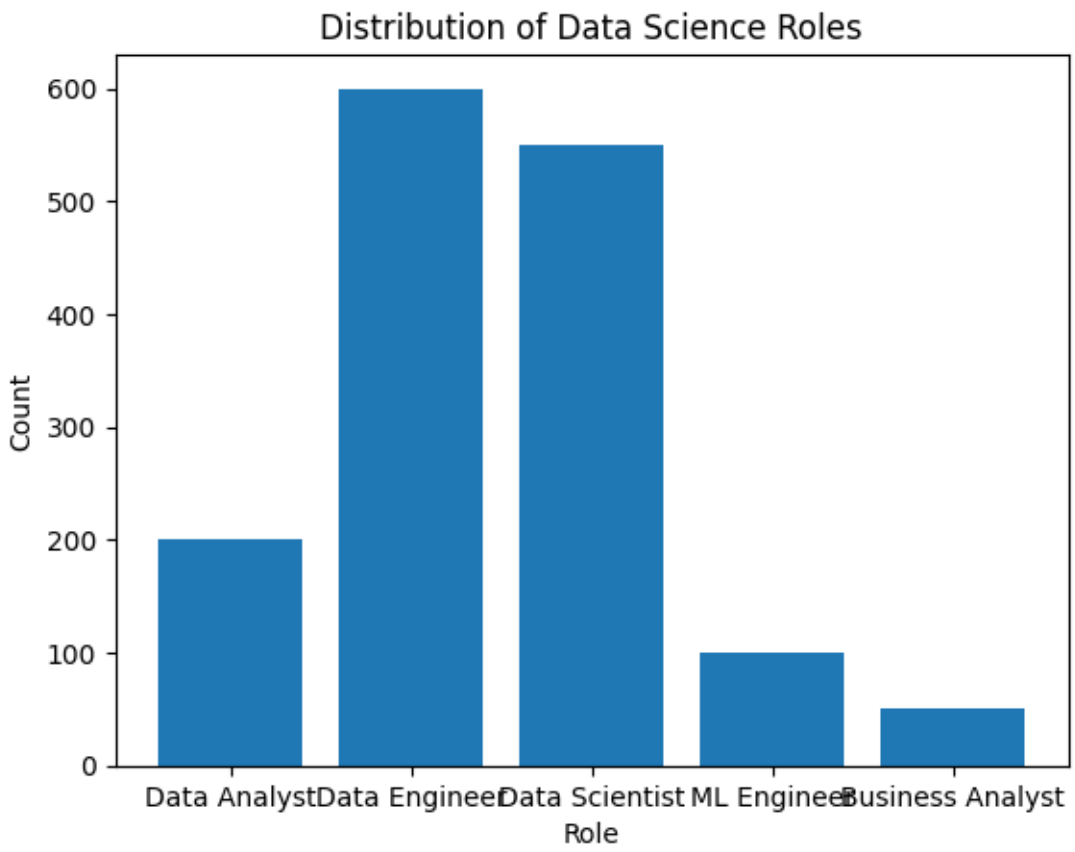
Description:

Use a dataset of job postings and categorize them into different roles. Visualize the distribution using pie charts or bar plots.

CODE:

```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer',
'Business Analyst']
counts = [200, 600, 550, 100, 50]
plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```

OUTPUT:



RESULT:

Data scientist is occurring rate in the dataset (count = s).Data analyst(3),Data Engineer(2),and ML Engineer(1) are equally data set.

Exp No:1.c Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

AIM:

Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Description:

Create small datasets for each type and explain their characteristics.

CODE:

```
structured_data = pd.DataFrame({
'ID': [2, 3, 4],
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [30, 35, 40]
})
print("Structured Data:\n", structured_data)
unstructured_data = "This is an example of unstructured data. It can be a piece of text, an image, or a video fiLe."
print("\nUnstructured Data:\n", unstructured_data)
semi_structured_data = {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
print("\nSemi-structured Data:\n", semi_structured_data)
```

OUTPUT:

Structured Data:

```
   ID    Name  Age
0  2    Alice   30
```

1 3 Bob 35

2 4 Charlie 40

Unstructured Data:

This is an example of unstructured data. It can be a piece of text, an image, or a video fiLe.

Semi-structured Data:

{'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}

RESULT:

The experiment was successfully perfomed by creating and displaying examples of structured,unstructured,semi structured data using python. The characteristics and representation of each type of data were observed successfully.

Exp No:1.d Conduct an experiment to encrypt and decrypt given sensitive data.

AIM:

Conduct an experiment to encrypt and decrypt given sensitive data.

Description:

Use the cryptography library to encrypt and decrypt a piece of data.

CODE:

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()

f = Fernet(key)

token = f.encrypt(b"Kennedy matric school")

token

b'...'

f.decrypt(token)

b'Kennedy matric school'

key = Fernet.generate_key()

cipher_suite = Fernet(key)

plain_text = b"Kennedy matric school."

cipher_text = cipher_suite.encrypt(plain_text)

decrypted_text = cipher_suite.decrypt(cipher_text)

print("Original Data:", plain_text)

print("Encrypted Data:", cipher_text)

print("Decrypted Data:", decrypted_text)
```

OUTPUT:

Original Data: b'Kennedy matric school.

Encrypted Data: b'gAAAAABqfHrH0QmZgXiVHtxa_aFrGkAt-h2wBRoKQBtoFWxfX6zwpEdPwmG5ef5oCxMpQGmoOz-ePuWJSxnLl0vGYMq_Z8FmI7M8zvQJVsp52N0ilzFWlzE='

Decrypted Data: b'Kennedy matric school.'

Exp No:2 Upload and Analyse the data set given in csv format and perform data preprocessing and visualization.

AIM:

Upload and Analyse the data set given in csv format and perform data preprocessing and visualization.

Description: Use sample data set sales-data.csv.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
file_path = 'sales-data-new.csv'
df = pd.read_csv(file_path)
print(df.head())
print(df.isnull().sum())
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
print(df.describe())
product_summary = df.groupby('Product').agg({
   'Sales': 'sum',
   'Quantity': 'sum'
}).reset_index()
print(product_summary)
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
```

```
plt.title('Total Sales by Product')
plt.show()

df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)

sales_over_time = df.groupby('Date').agg({
   'Sales': 'sum'
}).reset_index()

plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
plt.show()
pivot_table = df.pivot_table(
   values='Sales',
   index='Region',
   columns='Product',
   aggfunc='sum',
   fill_value=0
)

print(pivot_table)

# Correlation matrix
correlation_matrix = df[['Sales', 'Quantity']].corr()
```

```
print(correlation_matrix)

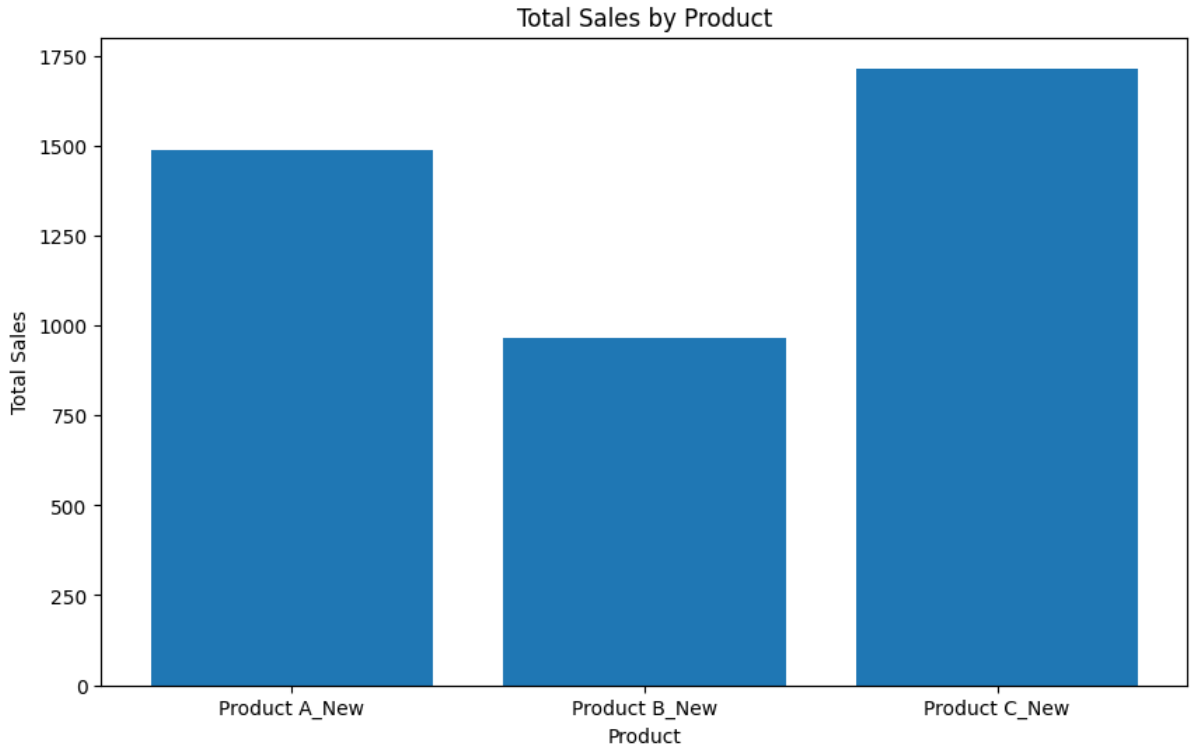

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

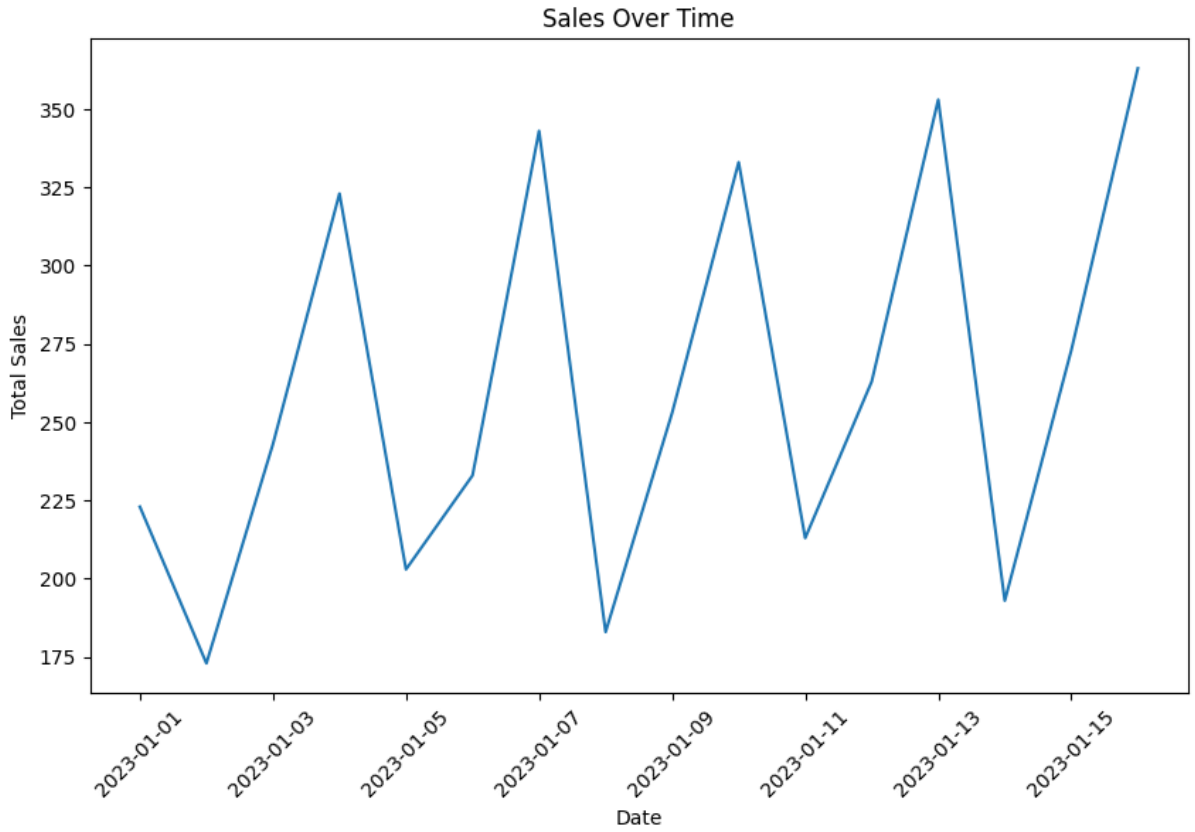
OUTPUT:

```
      Date      Product  Sales  Quantity    Region
0  01-01-2023  Product A_New    223       5  North_New
1  02-01-2023  Product B_New    173       4  South_New
2  03-01-2023  Product A_New    243       6  North_New
3  04-01-2023  Product C_New    323       7   East_New
4  05-01-2023  Product B_New    203       5   West_New
Date      0
Product    0
Sales      0
Quantity   0
Region     0
dtype: int64
        Sales   Quantity
count   16.000000  16.000000
mean   260.500000   6.375000
std    64.031242   1.746425
min    173.000000   4.000000
25%    210.500000   5.000000
50%    248.000000   6.500000
75%    325.500000   8.000000
```

max 363.000000 9.000000

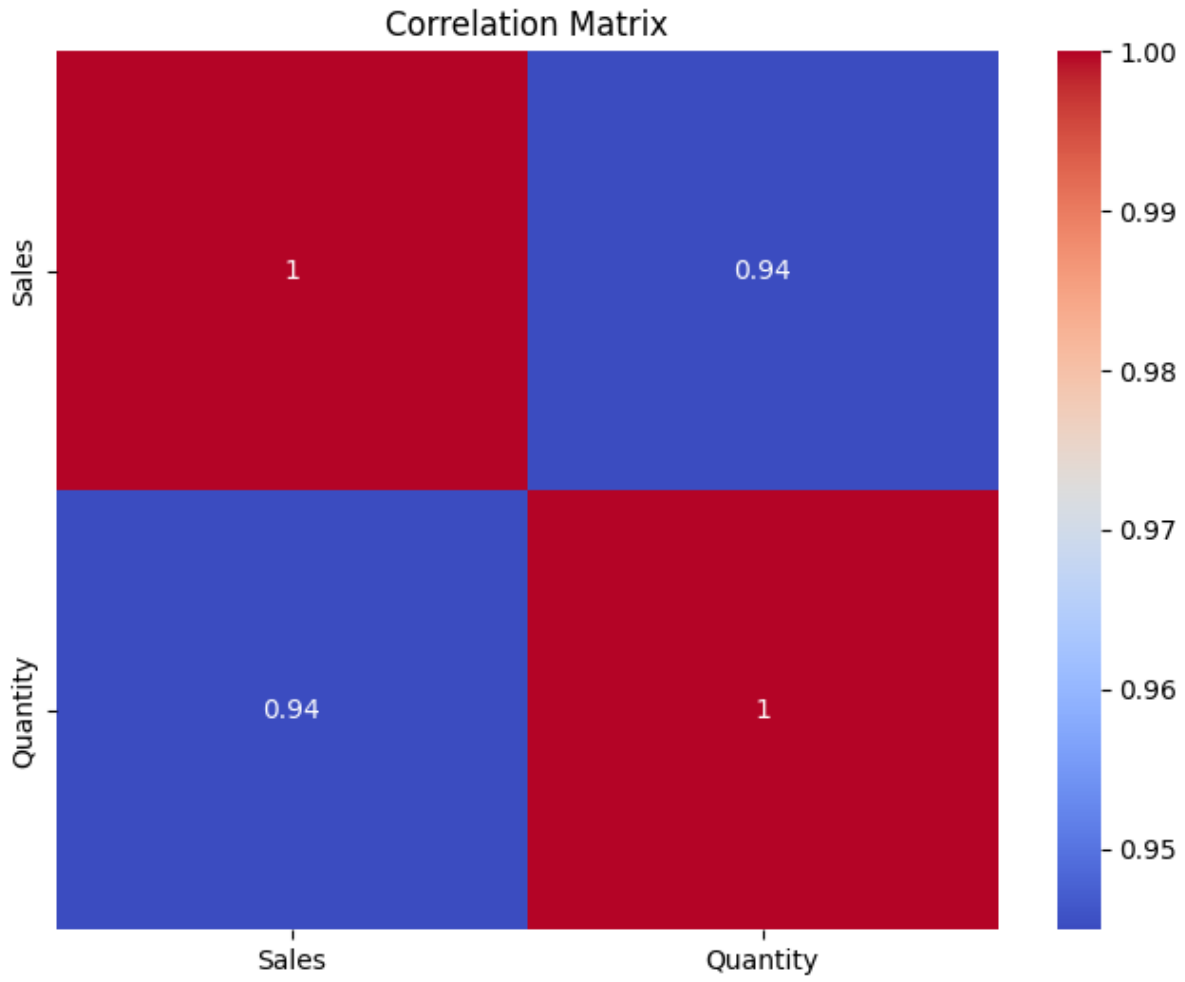
	Product	Sales	Quantity
0	Product A_New	1488	39
1	Product B_New	965	22
2	Product C_New	1715	41





Product	Product A_New	Product B_New	Product C_New
Region			
East_New	0	0	1715
North_New	1488	0	0
South_New	0	549	0
West_New	0	416	0

	Sales	Quantity
Sales	1.000000	0.944922
Quantity	0.944922	1.000000



RESULT:

The sales data was successfully analaysed and chart showed product-wise sales,sales trend,and the correlation between sales and quantity.

Exp No:3.a Conduct an experiment to show data visualization using line plot.

AIM:

Conduct an experiment to show data visualization using line plot.

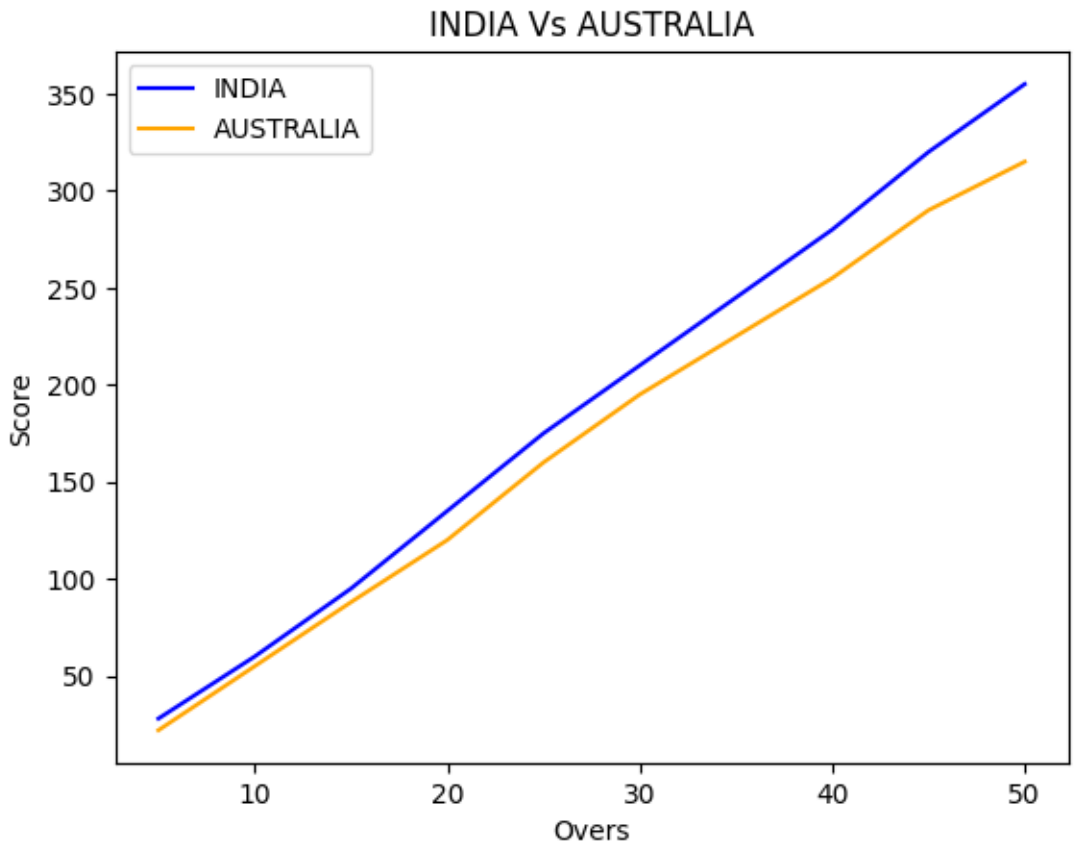
Description:

Take any sample data either through csv file or data fetched directly through code.

CODE:

```
import matplotlib.pyplot as plt
Overs = list(range(5, 51, 5))
India_Score = [28, 60, 95, 135, 175, 210, 245, 280, 320, 355]
Australia_Score = [22, 55, 88, 120, 160, 195, 225, 255, 290, 315]
plt.plot(Overs, India_Score, color="blue", label="INDIA")
plt.plot(Overs, Australia_Score, color="orange", label="AUSTRALIA")
plt.title("INDIA Vs AUSTRALIA")
plt.xlabel("Overs")
plt.ylabel("Score")
plt.legend(loc="upper left")
plt.show()
```

OUTPUT:



RESULT:

The line plot was successfully generated showing the score comparison between india and Australia over 50 years. The graph clearly ilustrate the scoring trends of both teams.

Exp No:3.b Conduct an experiment to show data visualization using bar chart.

AIM:

Conduct an experiment to show data visualization using bar chart.

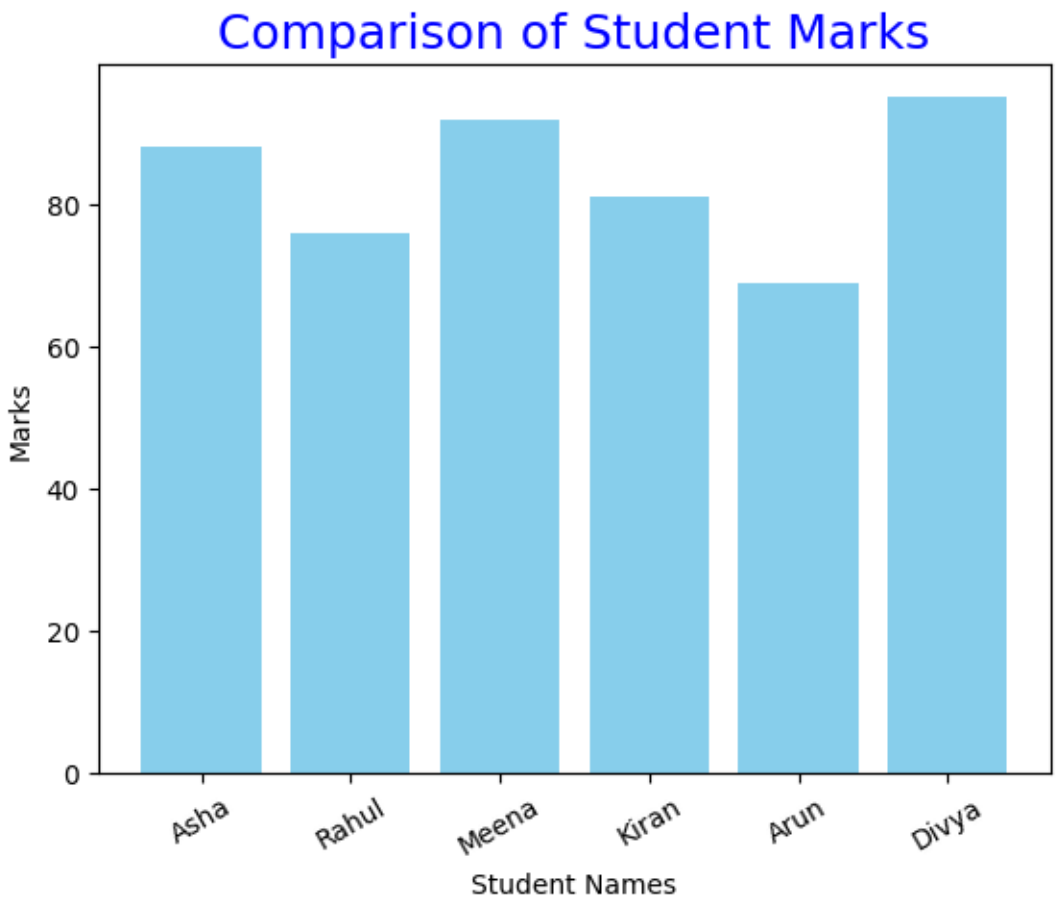
Description:

Take any sample data either through csv file or data fetched directly through code.

CODE:

```
import matplotlib.pyplot as plt
import numpy as np
Names = ['Asha', 'Rahul', 'Meena', 'Kiran', 'Arun', 'Divya']
Marks = [88, 76, 92, 81, 69, 95]
xaxis = np.arange(len(Names))
plt.bar(Names, Marks, color='skyblue')
plt.xticks(xaxis, Names, rotation=30)
plt.xlabel("Student Names")
plt.ylabel("Marks")
plt.title("Comparison of Student Marks", fontsize=18, color="blue")
plt.show()
```

OUTPUT:



RESULT:

The bar chart was successfully created using the given sample data.it clearly compares the marks obtained by student making the data easy to analyse and interpret usually.

Exp No:3.c Conduct an experiment to show data visualization using pie chart.

AIM:

Conduct an experiment to show data visualization using pie chart.

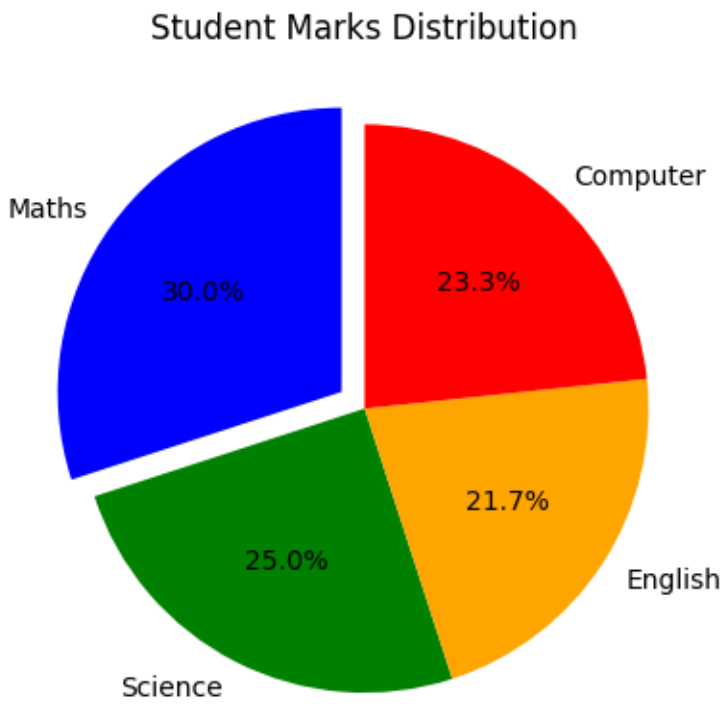
Description:

Take any sample data either through csv file or data fetched directly through code.

CODE:

```
import matplotlib.pyplot as plt
labels = ['Maths', 'Science', 'English', 'Tamil']
marks = [90, 75, 65, 70]
colors = ['blue', 'green', 'orange', 'red']
explode = (0.1, 0, 0, 0)
plt.pie(marks,
    labels=labels,
    colors=colors,
    explode=explode,
    autopct='%1.1f%%',
    startangle=90)
plt.title("Student Marks Distribution")
plt.show()
```

OUTPUT:



RESULT:

The experiment was successfully completed and the pie chart clearly represents the given data using proper lables,colors,proportions.

Exp No:4 To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

AIM:

To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

Description:

Import the word_tokenize function and gutenberg.

CODE:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')
sample = gutenberg.raw("carroll-alice.txt")
token = word_tokenize(sample)
wlist = []
for i in range(50):
  wlist.append(token[i])
wordfreq = [wlist.count(w) for w in wlist]
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

OUTPUT:

```
[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data]   Unzipping corpora/gutenberg.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

Pairs

[('[', 1), ('Alice', 2), ("'s", 1), ('Adventures', 1), ('in', 1), ('Wonderland', 1), ('by', 2), ('Lewis', 1), ('Carroll', 1), ('1865', 1), (']', 1), ('CHAPTER', 1), ('I', 1), ('.', 1), ('Down', 1), ('the', 3), ('Rabbit-Hole', 1), ('Alice', 2), ('was', 1), ('beginning', 1), ('to', 2), ('get', 1), ('very', 1), ('tired', 1), ('of', 2), ('sitting', 1), ('by', 2), ('her', 2), ('sister', 1), ('on', 1), ('the', 3), ('bank', 1), (',', 1), ('and', 1), ('of', 2), ('having', 1), ('nothing', 1), ('to', 2), ('do', 1), (':', 1), ('once', 1), ('or', 1), ('twice', 1), ('she', 1), ('had', 1), ('peeped', 1), ('into', 1), ('the', 3), ('book', 1), ('her', 2)]

RESULT:

The program successfully loads the sense and sensibility text,tokenizes it.select first 50 tokens and print each token along with its frequency.

Exp No:5 Data Collection and Initial Exploration

AIM:

To collect, load, and perform initial exploration of the diabetes dataset.

DESCRIPTION:

The diabetes dataset is loaded using pandas the dataset is explored using head(),info() and describe().histogram are used to visualize the distribution of each feature.

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"

columns = [

  'Pregnancies',

  'Glucose',

  'BloodPressure',

  'SkinThickness',

  'Insulin',

  'BMI',

  'DiabetesPedigreeFunction',

  'Age',

  'Outcome'

]

db = pd.read_csv(url, names=columns)

print(db.head())

print(db.info())

print(db.describe())

db.hist(bins=50, figsize=(20, 15))
```

```
plt.show()
sns.pairplot(db)
plt.show()
```

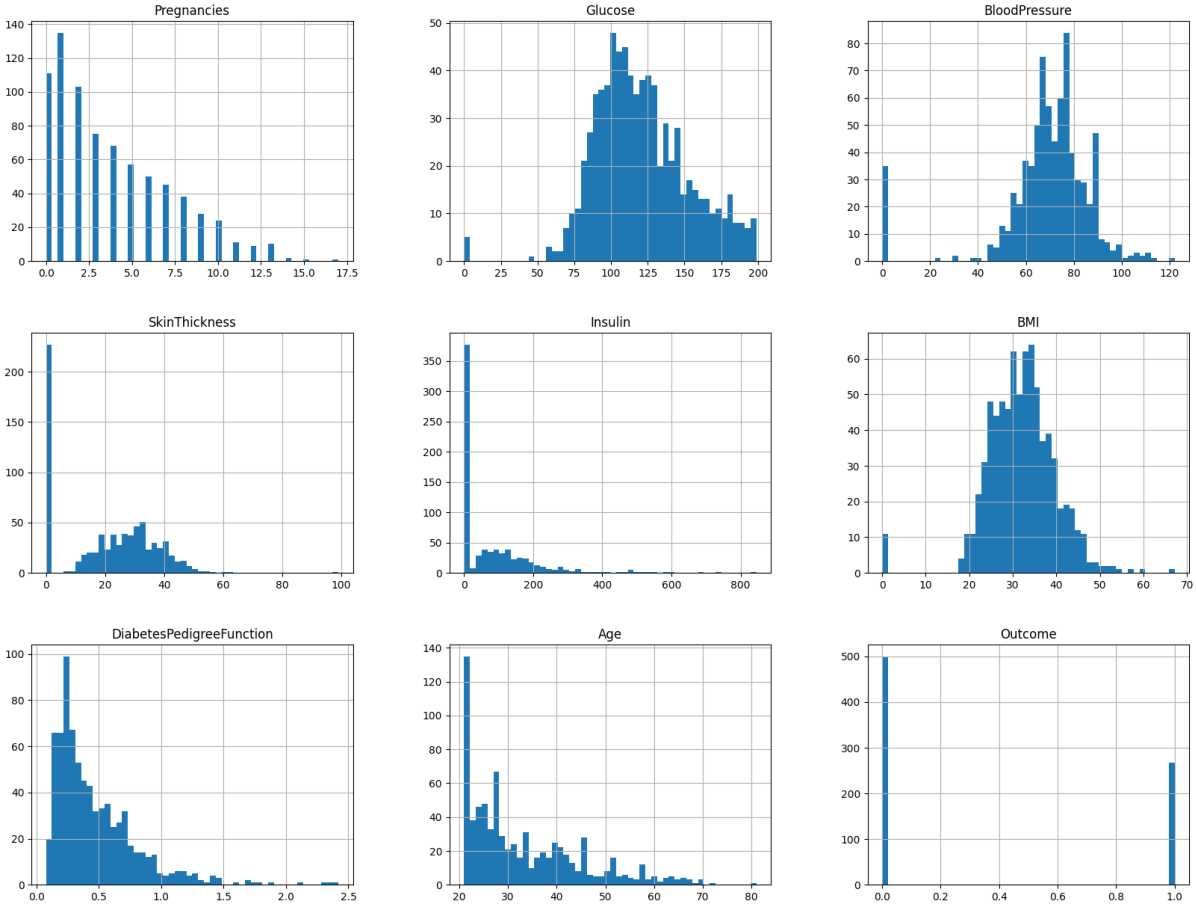
OUTPUT:

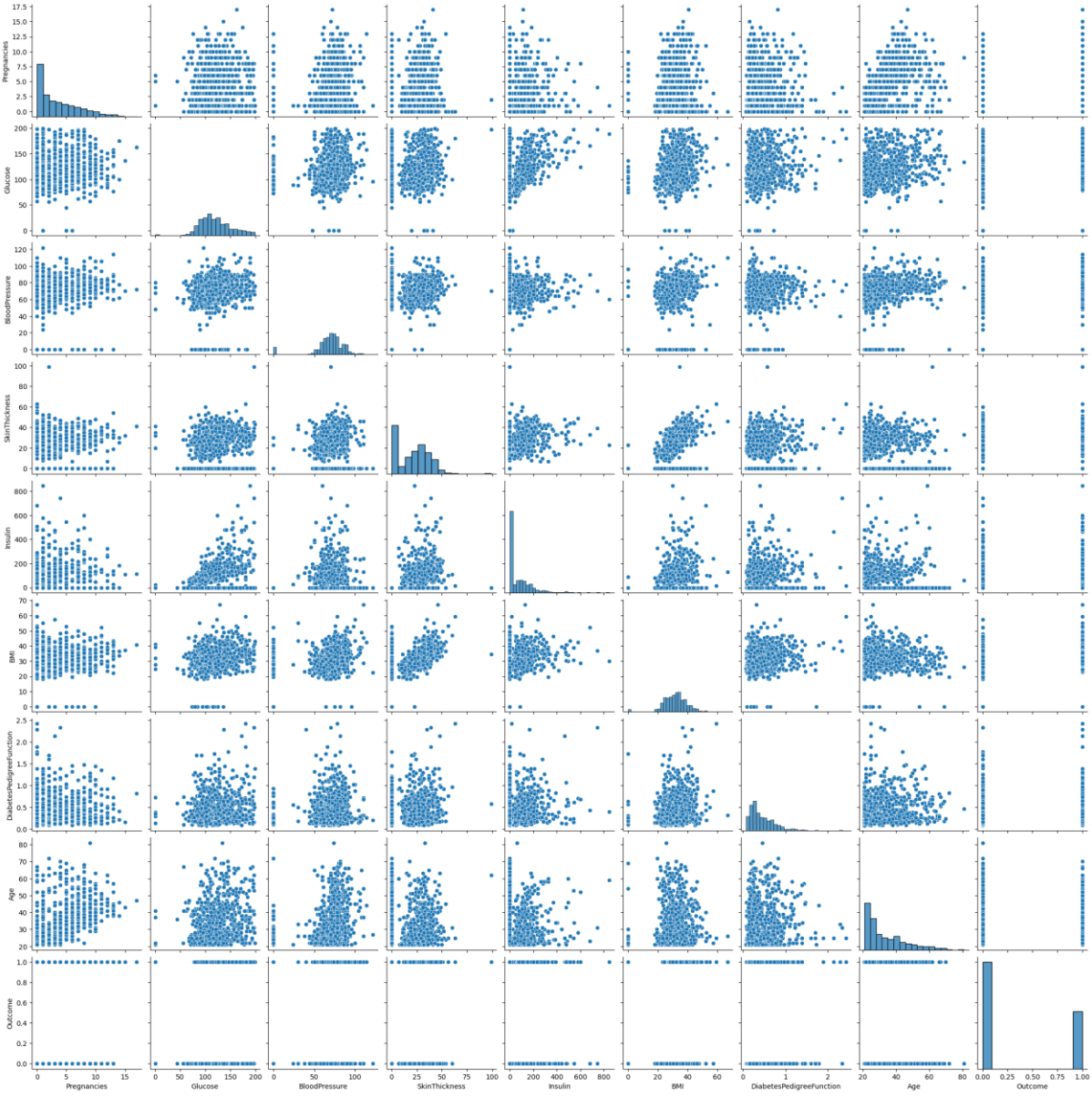
```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
```

```
 6  DiabetesPedigreeFunction  768 non-null    float64
 7  Age                       768 non-null    int64
 8  Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000             768.000000  768.000000  768.000000
mean    31.992578               0.471876   33.240885    0.348958
std      7.884160               0.331329   11.760232    0.476951
min      0.000000               0.078000   21.000000    0.000000
25%     27.300000               0.243750   24.000000    0.000000
50%     32.000000               0.372500   29.000000    0.000000
75%     36.600000               0.626250   41.000000    1.000000
max     67.100000               2.420000   81.000000    1.000000
```


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome



RESULT:

The diabetes dataset was successfully loaded and initially explored.the first few seconds,dataset structure and summary statistics were displayed.histogram showed the distribution of each feature and the pair plot showed the relationship between variables.

Experiment 6: Time Series Analysis of Website Traffic

Aim:

To analyze the time series data of website traffic to identify trends, seasonality,

and anomalies.

Description:

The website traffic dataset contains daily data for pageviews,unique visitors and bounce rate.The data is loaded and cleaned by converting the data column into a datetime format.Time series decomposition is used to identify trend , seasonal and residual components. A moving average is used to observe overall trend,and the z-score method is used to detect anomalies in page views.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from google.colab import files

uploaded = files.upload()

data = pd.read_csv("website_traffic_timeseries.csv")

data["Date"] = pd.to_datetime(data["Date"])

data.set_index("Date", inplace=True)

data.interpolate(method="linear", inplace=True)

print(data.head())

print(data.info())

print(data.describe())

plt.figure(figsize=(14, 6))


plt.plot(data.index, data["Page_Views"], label="Page Views")
```

```
plt.plot(data.index, data["Unique_Visitors"], label="Unique Visitors")
plt.plot(data.index, data["Bounce_Rate"], label="Bounce Rate")

plt.title("Website Traffic Over Time")
plt.xlabel("Date")
plt.ylabel("Values")
plt.legend()
plt.grid(True)
plt.show()
from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(
    data['Page_Views'],
    model='additive',
    period=7
)

decomposition.plot()
plt.tight_layout()
plt.show()
data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()

# Plot original data and moving average
plt.figure(figsize=(14, 6))

plt.plot(data.index, data["Page_Views"], label="Page Views")
```

```
plt.plot(data.index, data["Page_Views_MA"], label="30-Day Moving Average")

plt.title("Page Views with Moving Average")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.legend()
plt.grid(True)

plt.show()
data["Page_Views_Z"] = (
    (data["Page_Views"] - data["Page_Views"].mean())
    / data["Page_Views"].std()
)

# Identify anomalies
anomalies = data[data["Page_Views_Z"].abs() > 3]

print("Detected Anomalies:")
print(anomalies[["Page_Views", "Page_Views_Z"]])
plt.figure(figsize=(14, 6))

plt.plot(
    data.index,
    data["Page_Views"],
    label="Page Views"
)
```

```
# Highlight anomalies
plt.scatter(
    anomalies.index,
    anomalies["Page_Views"],
    color="red",
    label="Anomalies"
)

plt.title("Website Traffic Anomaly Detection")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.legend()
plt.grid(True)

plt.show()
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# Load dataset
data = pd.read_csv("website_traffic_timeseries.csv", parse_dates=["Date"],
index_col="Date")

# Handle missing values
data.interpolate(method="linear", inplace=True)
```

```
# Display first few rows
print(data.head())

# Dataset information and summary
print(data.info())
print(data.describe())

# Time Series Decomposition

from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(
    data["Page_Views"],
    model="additive",
    period=7
)

decomposition.plot()

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# Moving Average
data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()

data[["Page_Views", "Page_Views_MA"]].plot(figsize=(12, 5))
```

```
plt.title("Page Views with Moving Average")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.show()

# Anomaly Detection using Z-score
data["Page_Views_Z"] = (
    (data["Page_Views"] - data["Page_Views"].mean())
    / data["Page_Views"].std()
)

anomalies = data[data["Page_Views_Z"].abs() > 3]

print("Anomalies:")
print(anomalies[["Page_Views", "Page_Views_Z"]])

plt.figure(figsize=(12, 5))
plt.plot(data.index, data["Page_Views"], label="Page Views")
plt.scatter(
    anomalies.index,
    anomalies["Page_Views"],
    color="red",
    label="Anomalies"
)
plt.title("Website Traffic Anomaly Detection")
plt.xlabel("Date")
```

```
plt.ylabel("Page Views")

plt.legend()

plt.show()
```

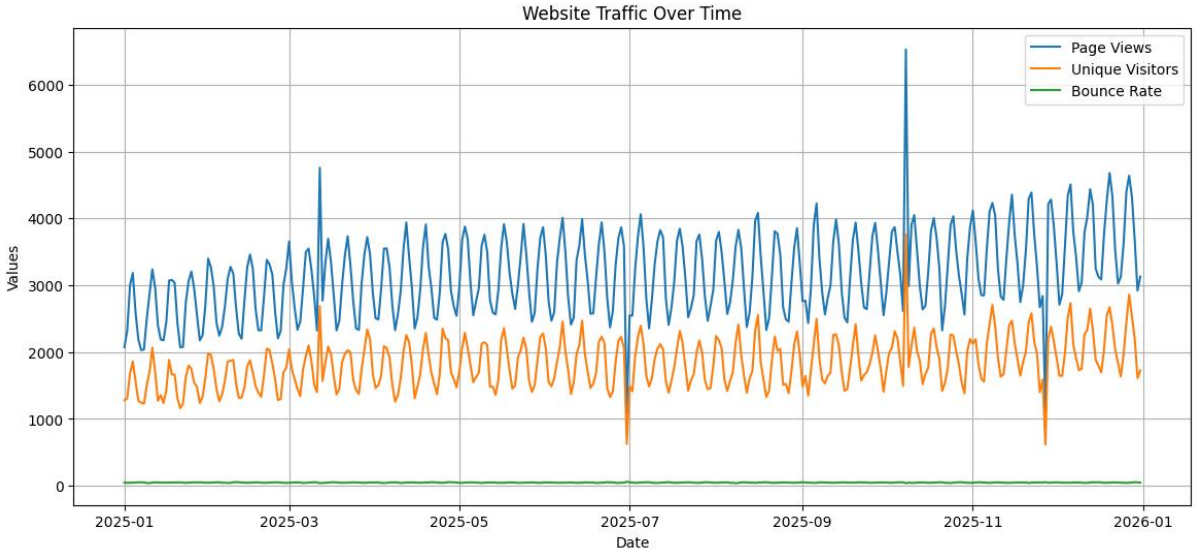
Output:

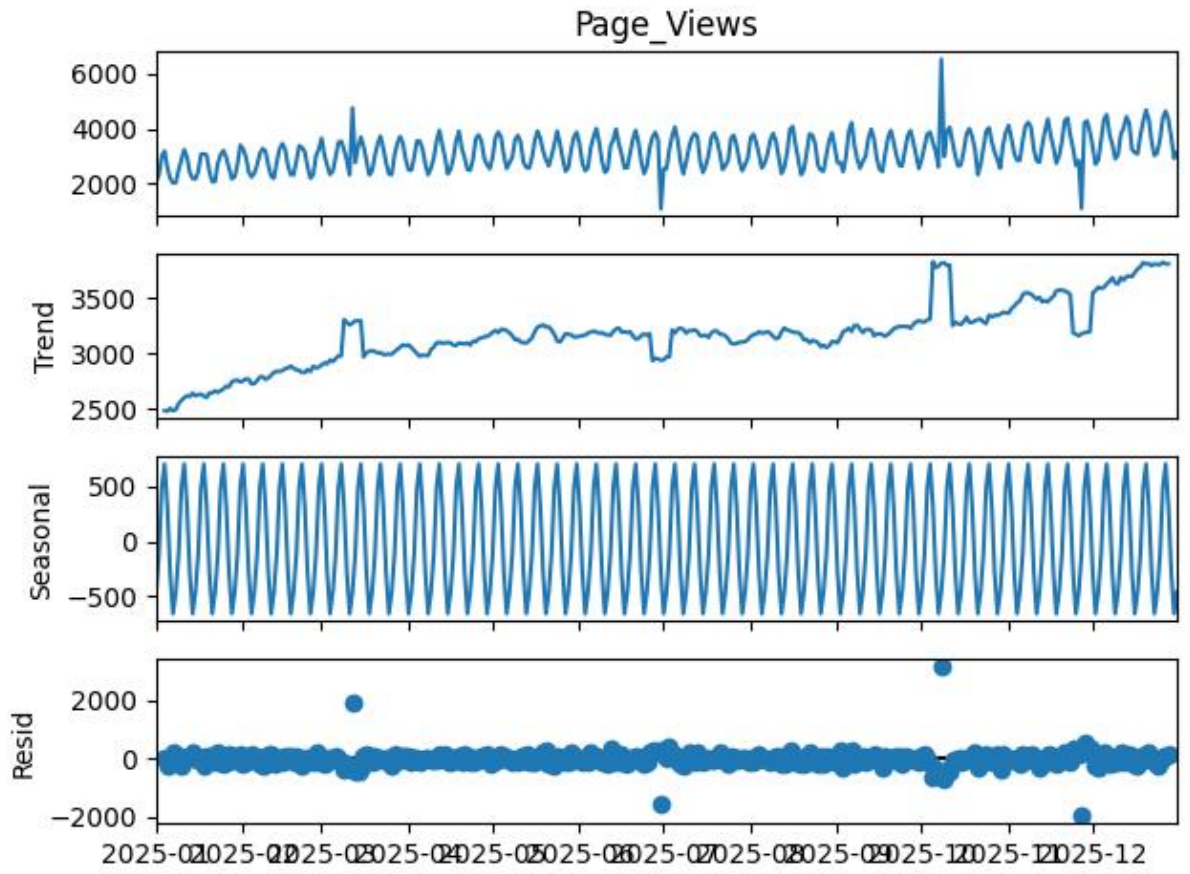
Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

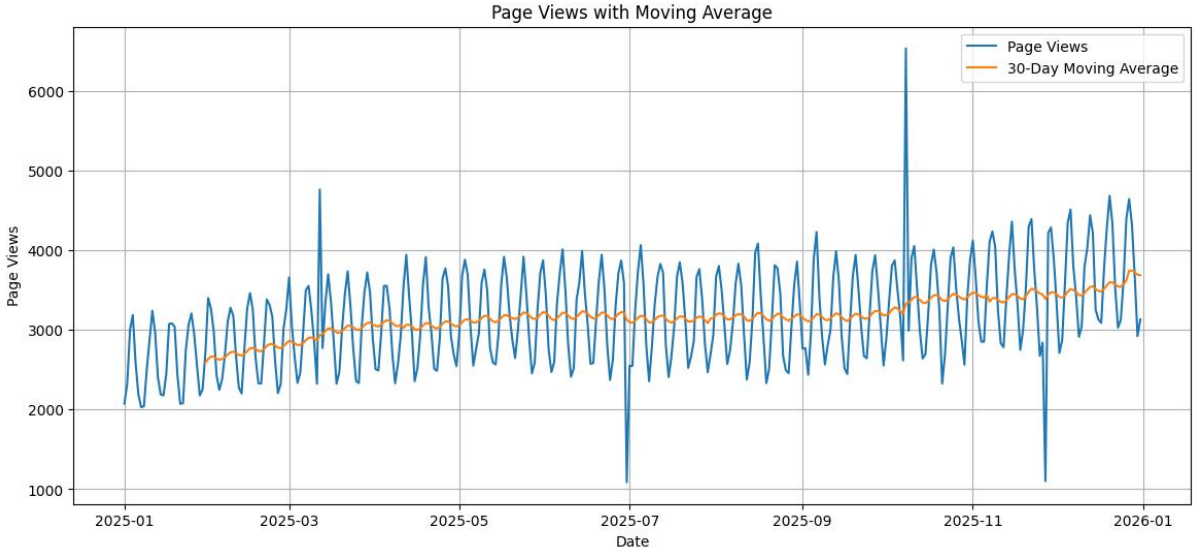
Saving website_traffic_timeseries.csv to website_traffic_timeseries.csv

```
            Page_Views  Unique_Visitors  Bounce_Rate
Date
2025-01-01        2074             1280        48.81
2025-01-02        2326             1306        44.98
2025-01-03        2999             1678        45.09
2025-01-04        3185             1863        45.93
2025-01-05        2586             1555        50.24
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 365 entries, 2025-01-01 to 2025-12-31
Data columns (total 3 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Page_Views      365 non-null    int64
 1   Unique_Visitors  365 non-null    int64
 2   Bounce_Rate     365 non-null    float64
dtypes: float64(1), int64(2)
memory usage: 11.4 KB
None
       Page_Views  Unique_Visitors  Bounce_Rate
count  365.000000       365.000000   365.000000
```

mean	3166.630137	1830.115068	48.036904
std	623.902005	371.898955	3.479127
min	1087.000000	617.000000	37.530000
25%	2686.000000	1550.000000	45.310000
50%	3105.000000	1797.000000	48.240000
75%	3614.000000	2083.000000	50.680000
max	6529.000000	3764.000000	60.550000



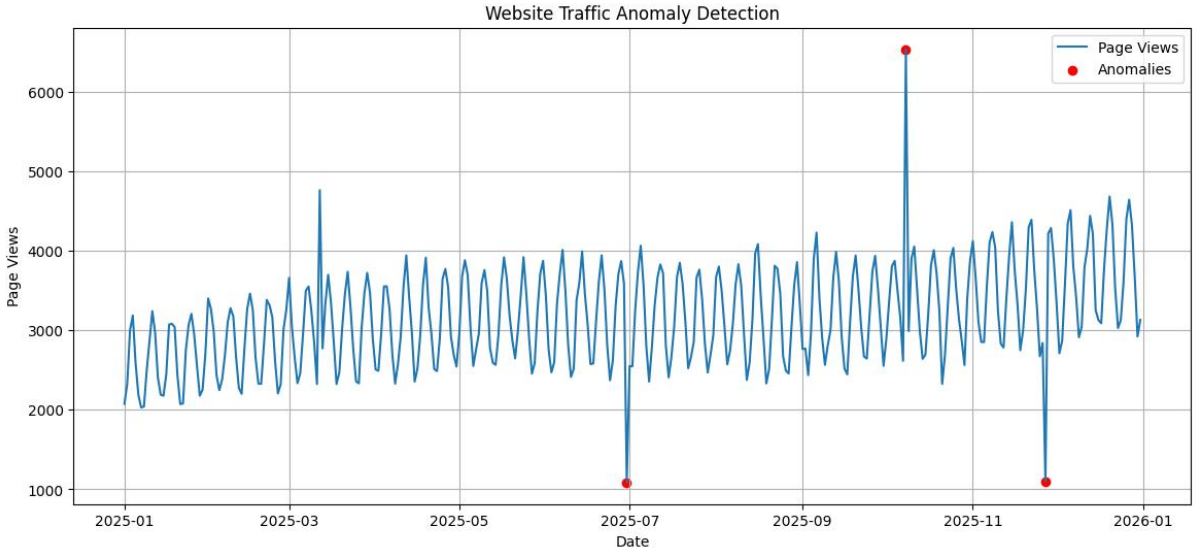




Detected Anomalies:

	Page_Views	Page_Views_Z
Date		
2025-06-30	1087	-3.333264
2025-10-08	6529	5.389260

2025-11-27 1101 -3.310825



	Page_Views	Unique_Visitors	Bounce_Rate
Date			
2025-01-01	2074	1280	48.81
2025-01-02	2326	1306	44.98
2025-01-03	2999	1678	45.09
2025-01-04	3185	1863	45.93
2025-01-05	2586	1555	50.24

<class 'pandas.core.frame.DataFrame'>

DatetimeIndex: 365 entries, 2025-01-01 to 2025-12-31

Data columns (total 3 columns):

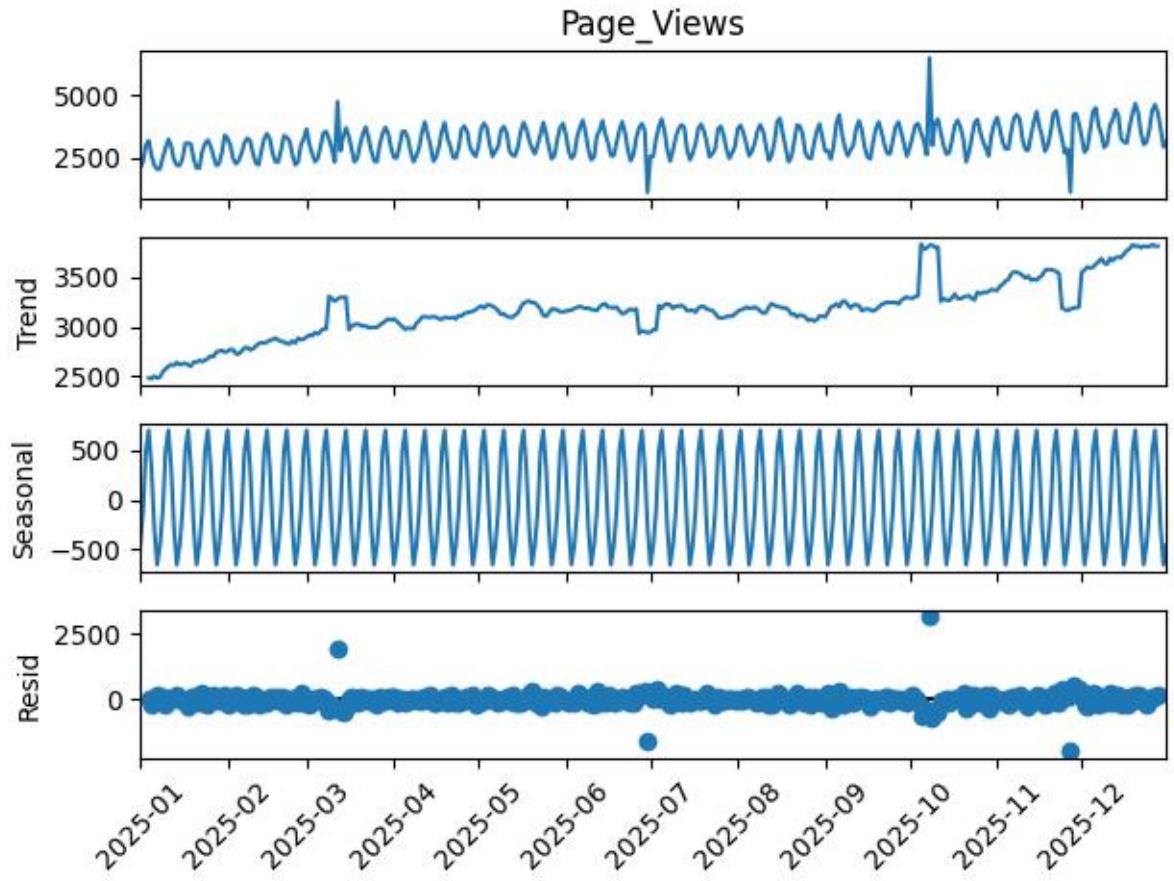
#	Column	Non-Null Count	Dtype
0	Page_Views	365 non-null	int64
1	Unique_Visitors	365 non-null	int64
2	Bounce_Rate	365 non-null	float64

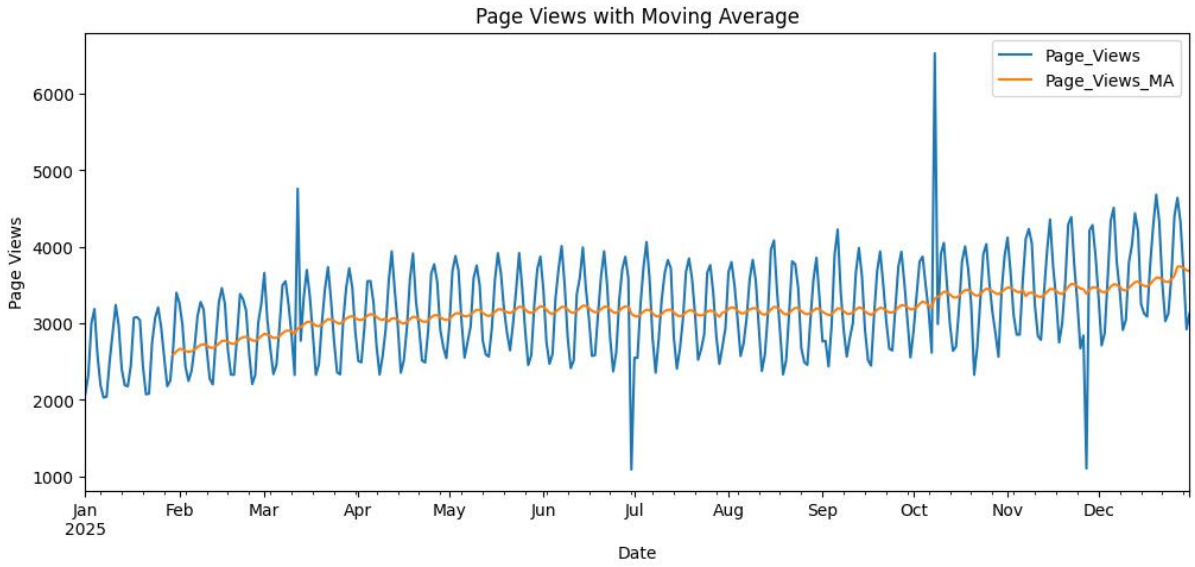
dtypes: float64(1), int64(2)

memory usage: 11.4 KB

None

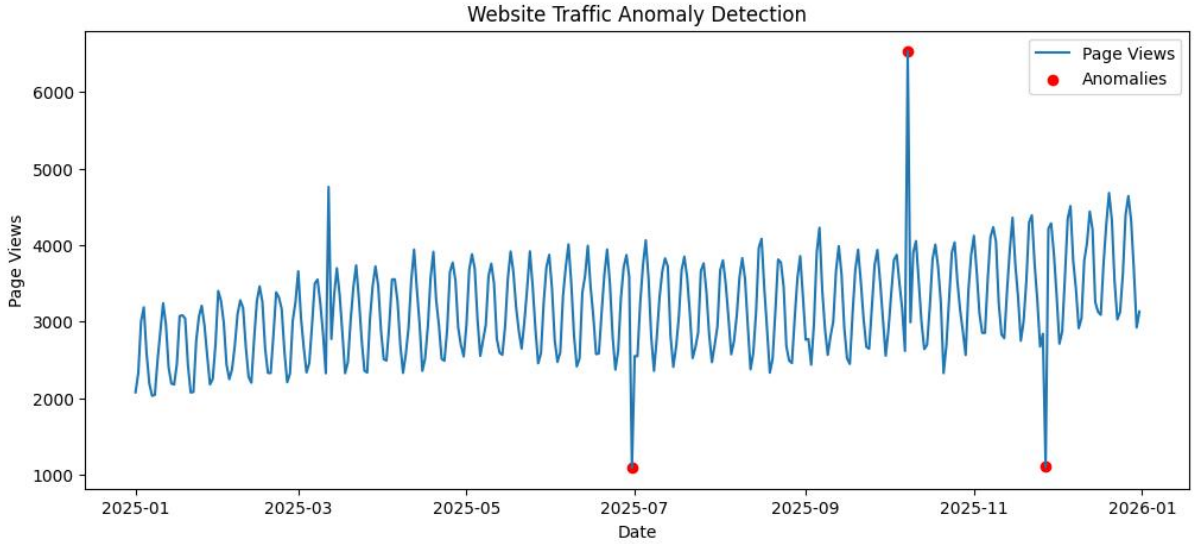
	Page_Views	Unique_Visitors	Bounce_Rate
count	365.000000	365.000000	365.000000
mean	3166.630137	1830.115068	48.036904
std	623.902005	371.898955	3.479127
min	1087.000000	617.000000	37.530000
25%	2686.000000	1550.000000	45.310000
50%	3105.000000	1797.000000	48.240000
75%	3614.000000	2083.000000	50.680000
max	6529.000000	3764.000000	60.550000





Anomalies:

	Page_Views	Page_Views_Z
Date		
2025-06-30	1087	-3.333264
2025-10-08	6529	5.389260
2025-11-27	1101	-3.310825



Result:

The website traffic over time is visualized using a line plot.showing the variations and sudden increases in page view.

Experiment 7:. Random Sampling and Sampling Distribution

AIM:

To study random sampling and understand the sampling distribution of sample means using python

DESCRIPTION:

A population is generated and random samples of different sizes are selected.the samples means are calculated and plotted to observe how the sampling distribution changes with sample size.

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

population_mean = 75

population_std = 12

population_size = 40000

population = np.random.normal(population_mean, population_std, population_size)

sample_sizes = [15, 40, 80]

num_samples = 300

sample_means = {}

for size in sample_sizes:

  sample_means[size] = []

  for _ in range(num_samples):

    sample = np.random.choice(population, size=size, replace=False)

    sample_means[size].append(np.mean(sample))

plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):

  plt.subplot(len(sample_sizes), 1, i + 1)

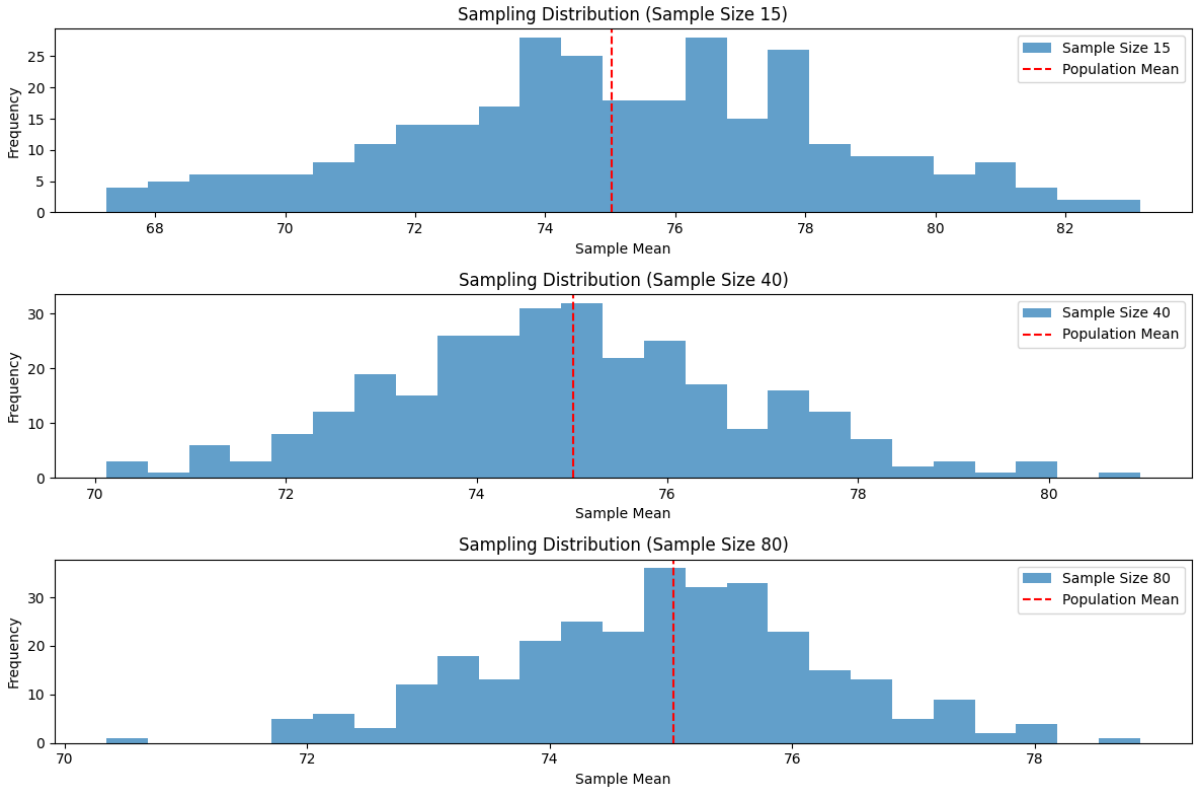
  plt.hist(sample_means[size], bins=25, alpha=0.7,

       label=f'Sample Size {size}')

  plt.axvline(np.mean(population), color='
```

```
    linestyle='dashed', linewidth=1.5,
            label='Population Mean')
    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()
plt.tight_layout()
plt.show()
```

OUTPUT:



RESULT:

Random sampling and sample distributions were successfully obtained and analysed.

Lab Experiment 8: Conduct Z test for the Data Given using Python Objective:

AIM:

To test whether the average weight of a species of birds differs from 150 grams.

DESCRIPTION:

1. Null Hypothesis (H0): The average weight of the birds is 150 grams.

2. Alternative Hypothesis (H1): The average weight of the birds is not 150 grams.

3. Sample: Measure the weights of 30 birds randomly selected from the population.

4. Z-Test: Conduct a Z-test to compare the sample mean to 150 grams.

5. Decision Rule: Use a significance level of α = 0.05.

CODE:

```
import numpy as np
import scipy.stats as stats
sample_data = np.array([
   145, 148, 150, 147, 149, 151, 146, 152,
   150, 148, 149, 147, 153, 150, 151, 146,
   149, 152, 148, 150, 147, 151, 149, 150,
   146, 153, 148, 151, 150, 149
])
population_mean = 150
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)
n = len(sample_data)
z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))
p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))
print("Sample Mean:", sample_mean)
print("Sample Standard Deviation:", sample_std)
print("Z-statistic:", z_statistic)
print("P-value:", p_value
```

```
if p_value < 0.05:
    print("Reject the null hypothesis")
else:
    print("Fail to reject the null hypothesis")
```

OUTPUT:

Sample Mean: 149.16666666666666

Sample Standard Deviation: 2.118609389936341

Z-statistic: -2.154410656139678

P-value: 0.031207981635716475

Reject the null hypothesis

RESULT:

There is no significant difference between the average bird weight and 150 grams at the 5% significance level

Exp No:9) Matplotlib library in data science.

Aim:

To understand the use of the matplotlib library in data science through visualization.

Description:

Use Iris data set to understand the Matplot lib library

Code:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
data = pd.read_excel('/content/Social_Network_Ads.xlsx')
data
data.info()
data.describe()
data.value_counts('Gender')
dummies = pd.get_dummies(data.Gender)
FinalDataset = pd.concat(
   [dummies, data.iloc[:, [0, 2, 3, 4]]],
   axis=1
)
FinalDataset.head()
sns.scatterplot(
   x='Age',
   y='EstimatedSalary',
   hue='Gender',
   data=data
)
```

```
plt.show()
sns.scatterplot(
   x='Age',
   y='EstimatedSalary',
   hue='Purchased',
   data=data
)

plt.show()
g = sns.pairplot(
   data[['Age', 'EstimatedSalary', 'Purchased']],
   hue='Purchased',
   height=3
)
for ax in g.axes.flat:
   if ax is not None:
      ax.tick_params(axis='x', labelrotation=45)
plt.show()
sns.FacetGrid(
   data,
   hue='Gender',
   height=5
).map(
   sns.histplot,
   'EstimatedSalary'
).add_legend()

plt.show()
```

```
sns.FacetGrid(
   data,
   hue='Gender',
   height=5
).map(
   sns.histplot,
   'Age'
).add_legend()

plt.show()
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
data = pd.read_excel('/content/Social_Network_Ads.xlsx')
data
data.info()
data.describe()
data.value_counts('Gender')
sns.countplot(x='Gender', data=data)
plt.show()
dummies = pd.get_dummies(data.Gender)

FinalDataset = pd.concat(
   [dummies, data.iloc[:, [0, 2, 3, 4]]],
   axis=1
)
FinalDataset.head()
```

```
sns.scatterplot(
   x='Age',
   y='EstimatedSalary',
   hue='Gender',
   data=data
)
plt.show()
sns.scatterplot(
   x='Age',
   y='EstimatedSalary',
   hue='Purchased',
   data=data
)
plt.show()
g = sns.pairplot(
   data[['Age', 'EstimatedSalary', 'Purchased']],
   hue='Purchased',
   height=3
)
for ax in g.axes.flat:
   if ax is not None:
      ax.tick_params(axis='x', labelrotation=45)

plt.show()
sns.FacetGrid(
   data,
   hue='Gender',
   height=5
```

```
).map(
   sns.histplot,
   'EstimatedSalary'
).add_legend()

plt.show()
sns.FacetGrid(
   data,
   hue='Gender',
   height=5
).map(
   sns.histplot,
   'Age'
).add_legend()

plt.show()
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
data = pd.read_excel('/content/Social_Network_Ads.xlsx')
data
data.info()
data.describe()
data.value_counts('Gender')
sns.countplot(x='Gender', data=data)
plt.show()
dummies = pd.get_dummies(data.Gender)
```

```
FinalDataset = pd.concat(
   [dummies, data.iloc[:, [0, 2, 3, 4]]],
   axis=1
)

FinalDataset.head()

sns.scatterplot(
   x='Age',
   y='EstimatedSalary',
   hue='Gender',
   data=data
)
plt.show()
sns.scatterplot(
   x='Age',
   y='EstimatedSalary',
   hue='Purchased',
   data=data
)
plt.show()

g = sns.pairplot(
   data[['Age', 'EstimatedSalary', 'Purchased']],
   hue='Purchased',
   height=3
)
```

```
for ax in g.axes.flat:
    if ax is not None:
        ax.tick_params(axis='x', labelrotation=45)

plt.show()
sns.FacetGrid(
    data,
    hue='Gender',
    height=5
).map(
    sns.histplot,
    'EstimatedSalary'
).add_legend()

plt.show()
sns.FacetGrid(
    data,
    hue='Gender',
    height=5
).map(
    sns.histplot,
    'Age'
).add_legend()

plt.show()
```

Output:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 400 entries, 0 to 399

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	User ID	400 non-null	int64
1	Gender	400 non-null	object
2	Age	400 non-null	int64
3	EstimatedSalary	400 non-null	int64
4	Purchased	400 non-null	int64

dtypes: int64(4), object(1)

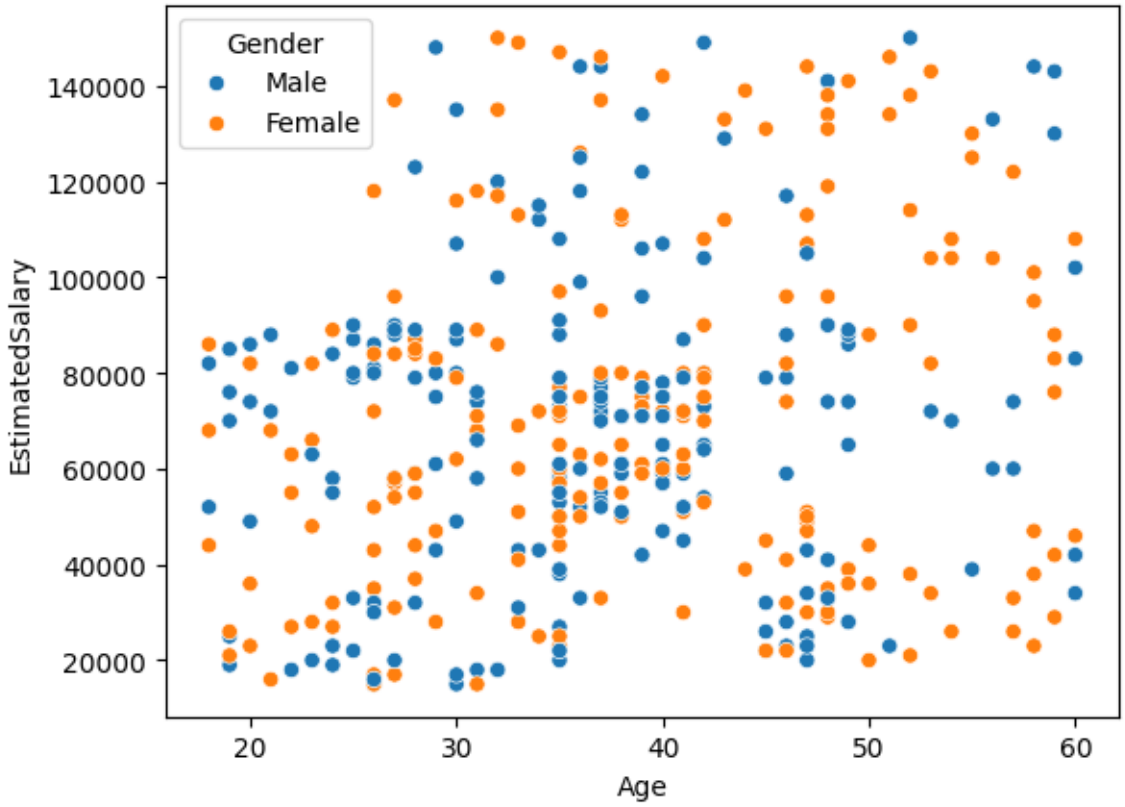
memory usage: 15.8+ KB

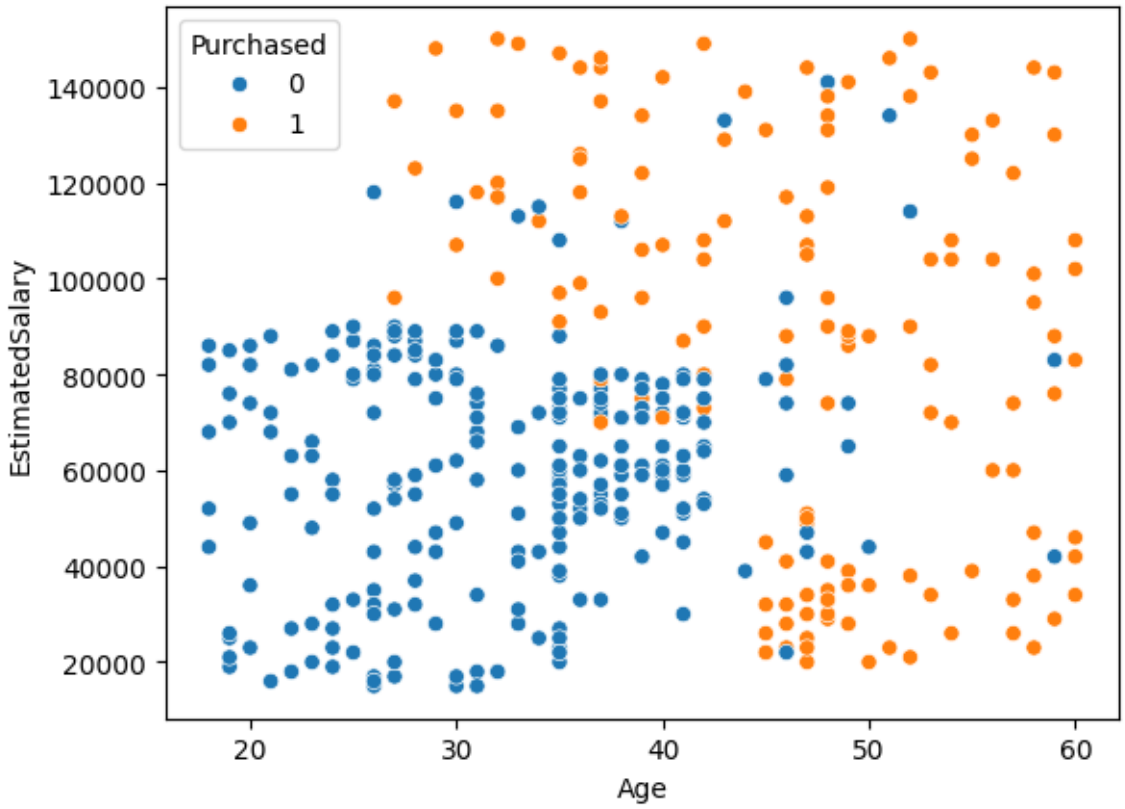
	User ID	Age	EstimatedSalary	Purchased
count	4.000000e+02	400.000000	400.000000	400.000000
mean	1.569154e+07	37.655000	69742.500000	0.357500
std	7.165832e+04	10.482877	34096.960282	0.479864
min	1.556669e+07	18.000000	15000.000000	0.000000
25%	1.562676e+07	29.750000	43000.000000	0.000000
50%	1.569434e+07	37.000000	70000.000000	0.000000
75%	1.575036e+07	46.000000	88000.000000	1.000000
max	1.581524e+07	60.000000	150000.000000	1.000000

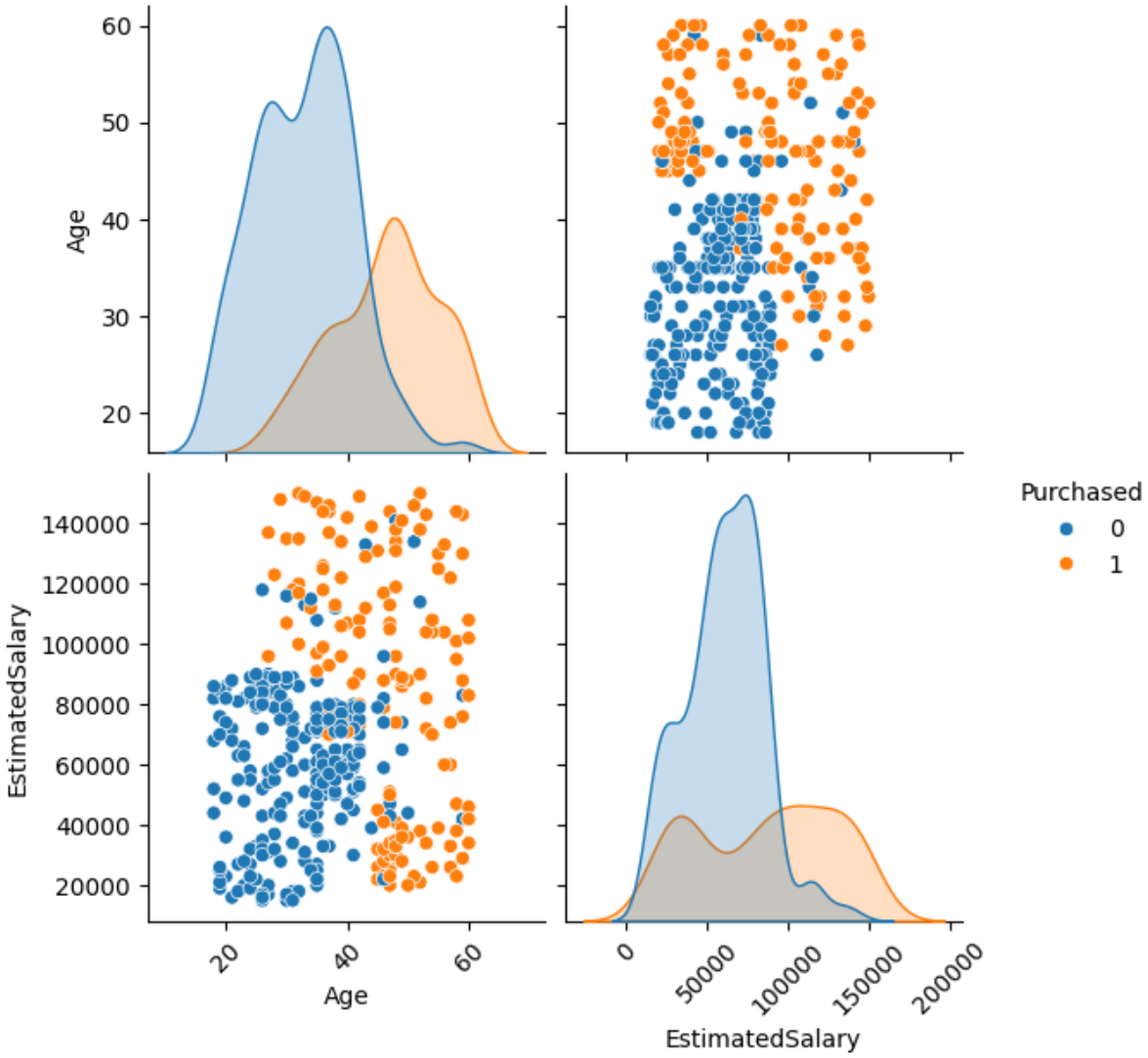
	count
Gender	
Female	204
Male	196

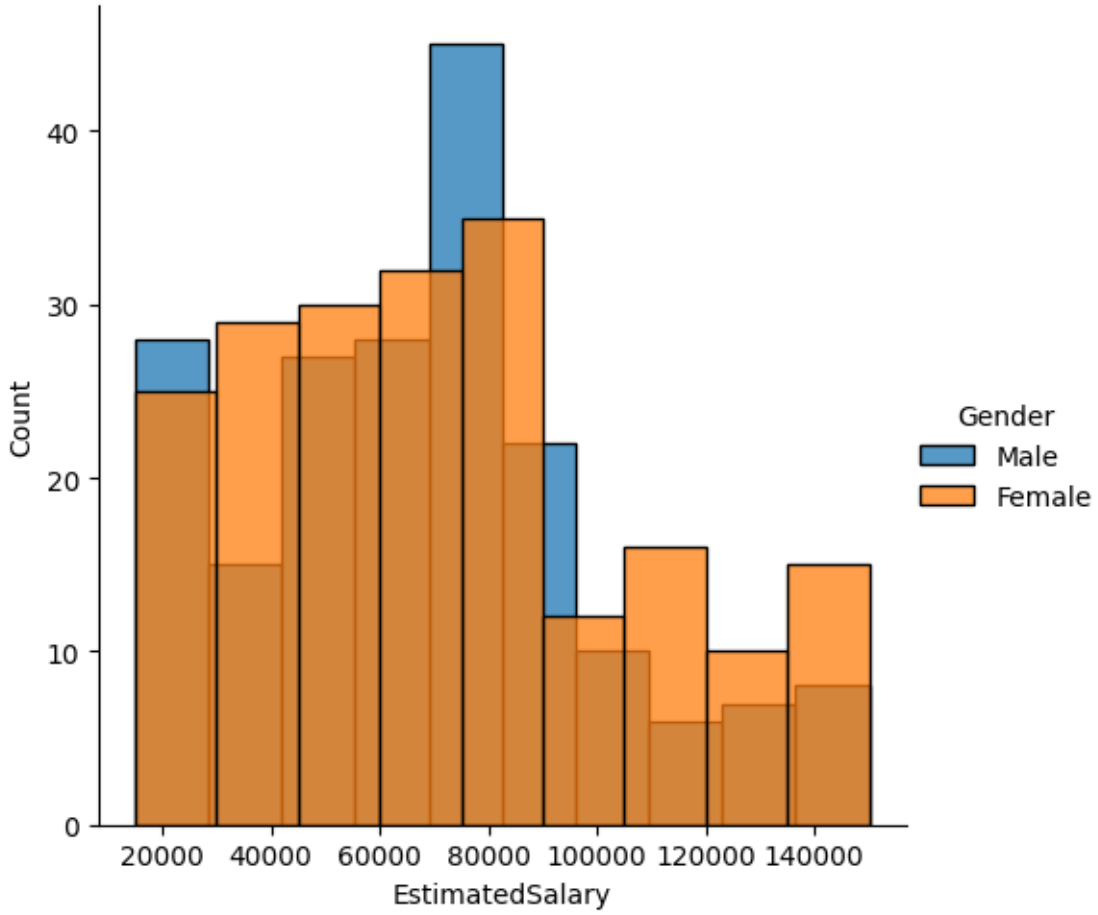
dtype: int64

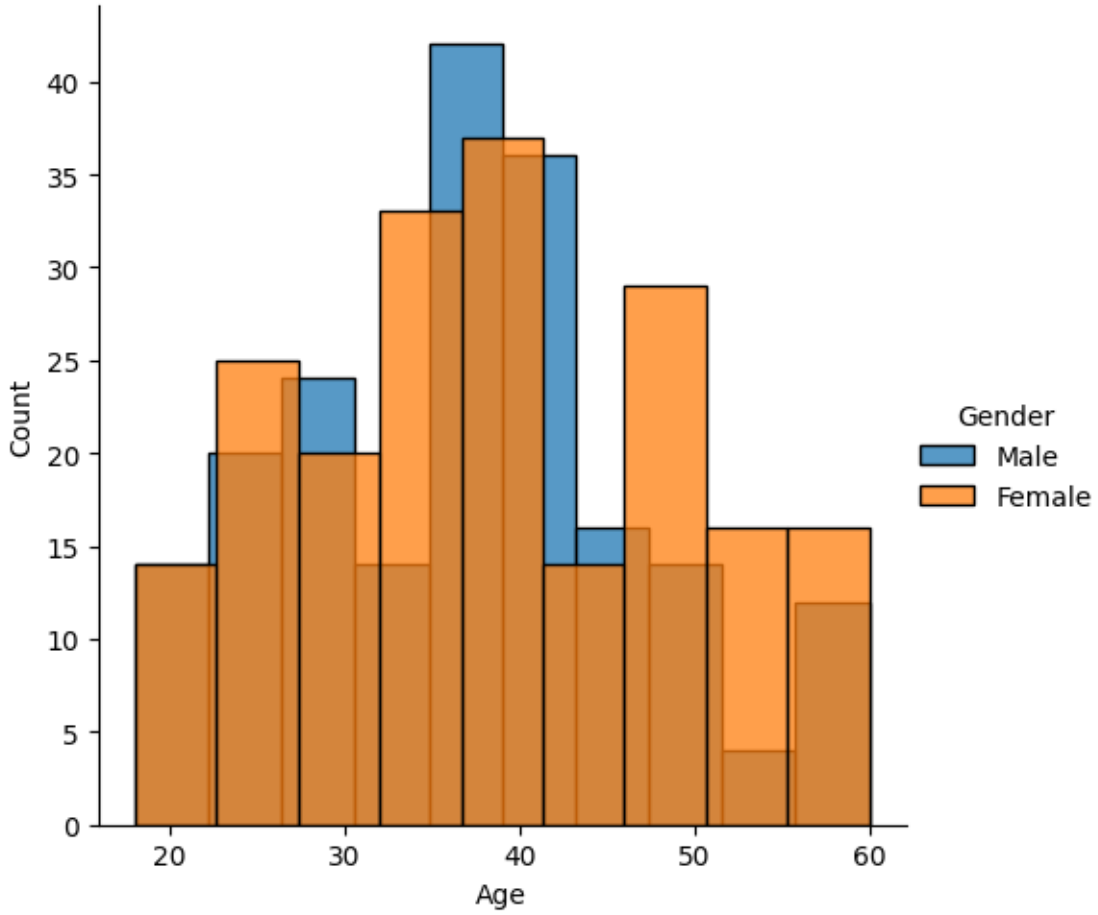
	Female	Male	User ID	Age	EstimatedSalary	Purchased
0	False	True	15624510	19	19000	0
1	False	True	15810944	35	20000	0
2	True	False	15668575	26	43000	0
3	True	False	15603246	27	57000	0
4	False	True	15804002	19	76000	0


Gender
Male
Female
EstimatedSalary
140000
120000
100000
80000
60000
40000
20000
20
30
40
50
60
Age


Purchased
0
1
EstimatedSalary
140000
120000
100000
80000
60000
40000
20000
20
30
40
50
60
Age


Age
EstimatedSalary
Purchased
0
1
60
50
40
30
20
140000
120000
100000
80000
60000
40000
20000
20
40
60
0
50000
100000
150000
200000


Count
0
10
20
30
40
20000
40000
60000
80000
100000
120000
140000
EstimatedSalary
Gender
Male
Female



<class 'pandas.core.frame.DataFrame'>

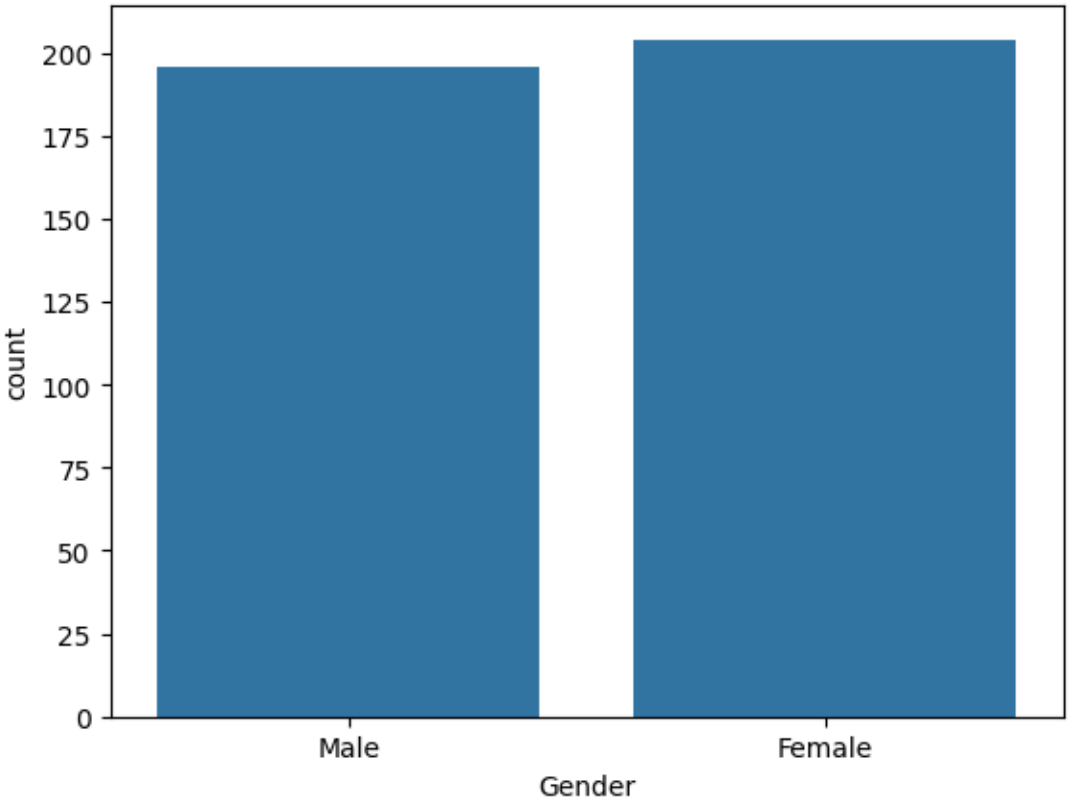
RangeIndex: 400 entries, 0 to 399

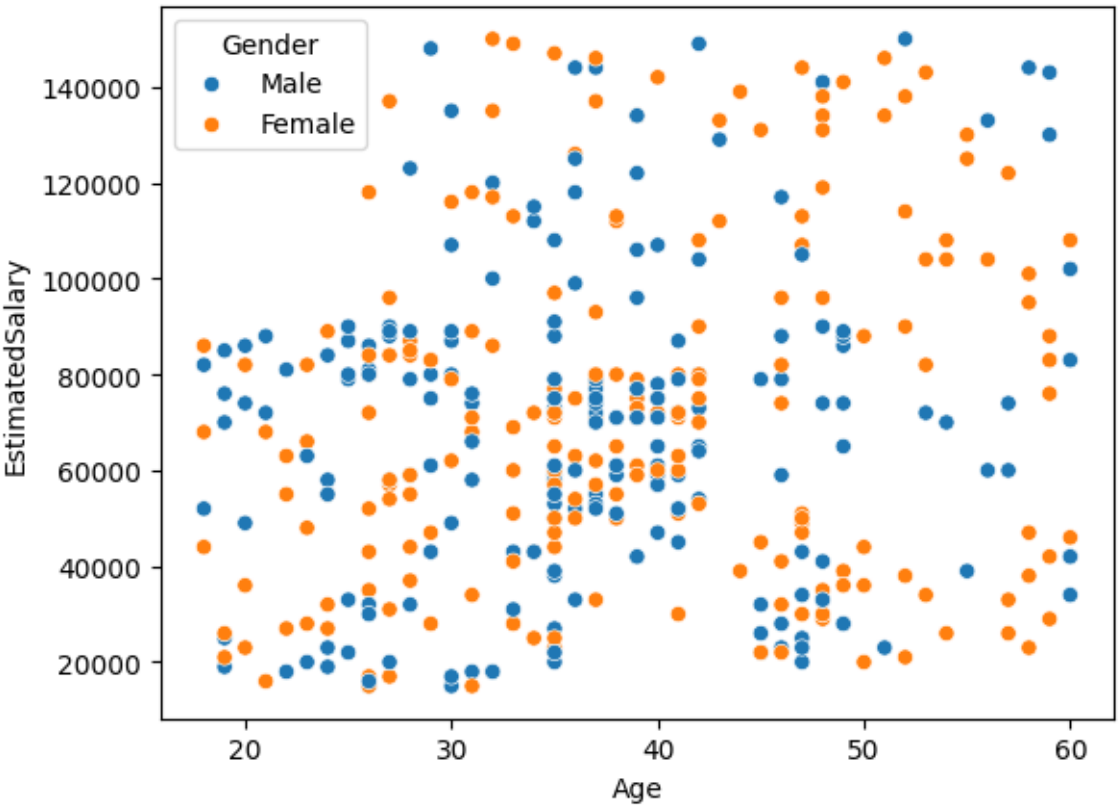
Data columns (total 5 columns):

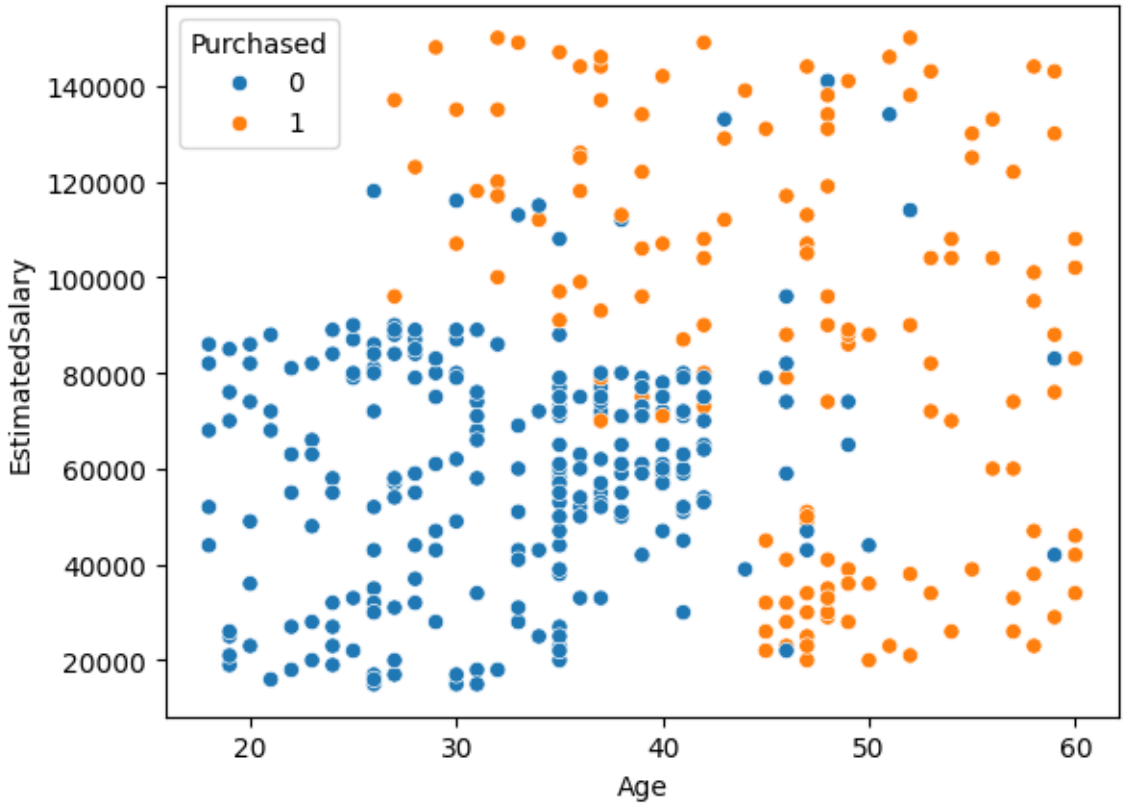
#	Column	Non-Null Count	Dtype
0	User ID	400 non-null	int64
1	Gender	400 non-null	object
2	Age	400 non-null	int64
3	EstimatedSalary	400 non-null	int64
4	Purchased	400 non-null	int64

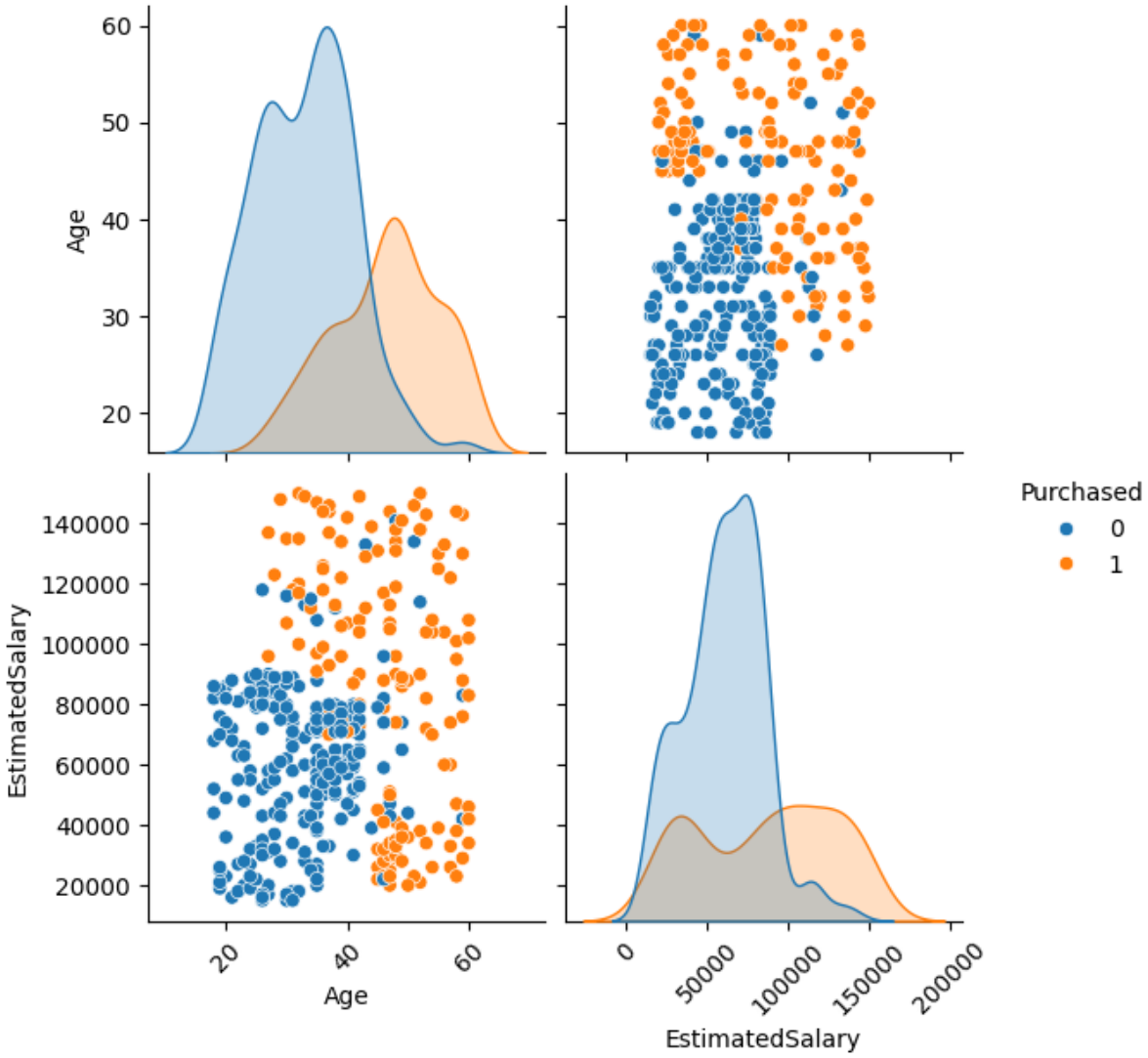
dtypes: int64(4), object(1)

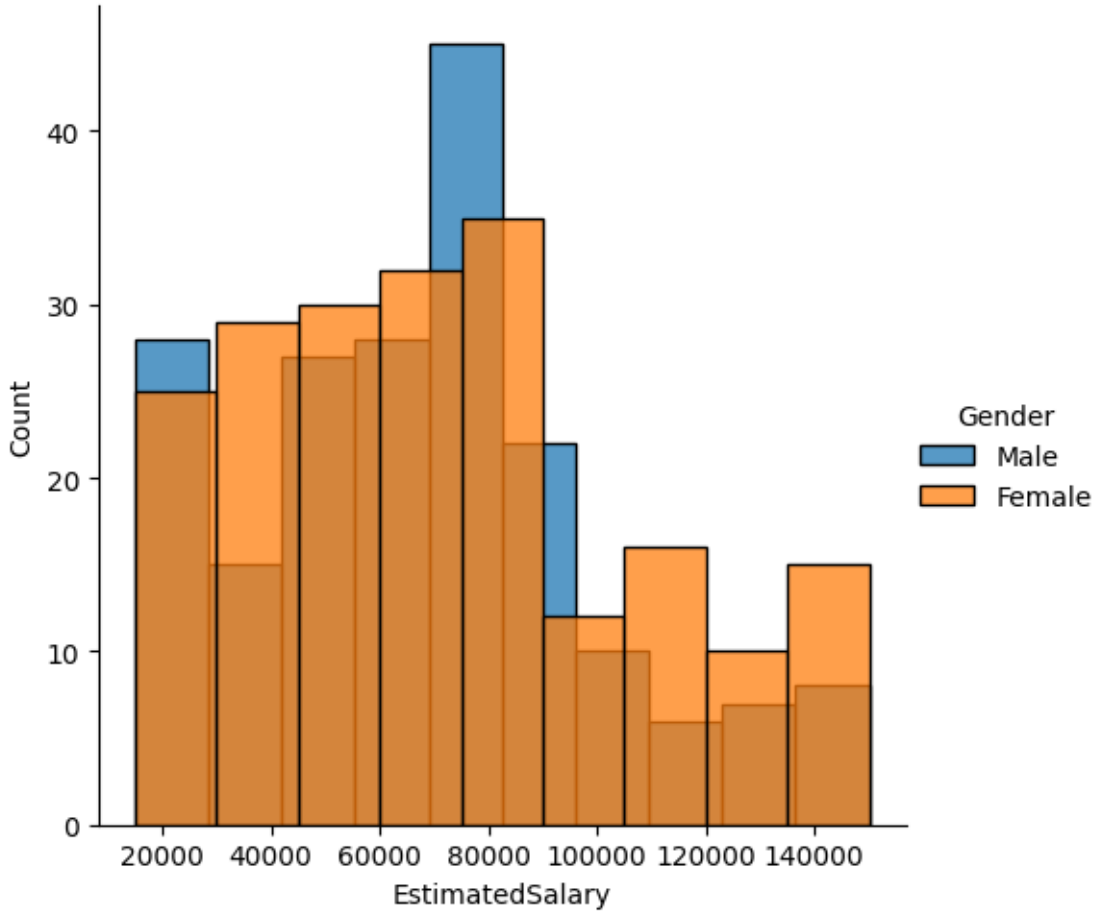
memory usage: 15.8+ KB

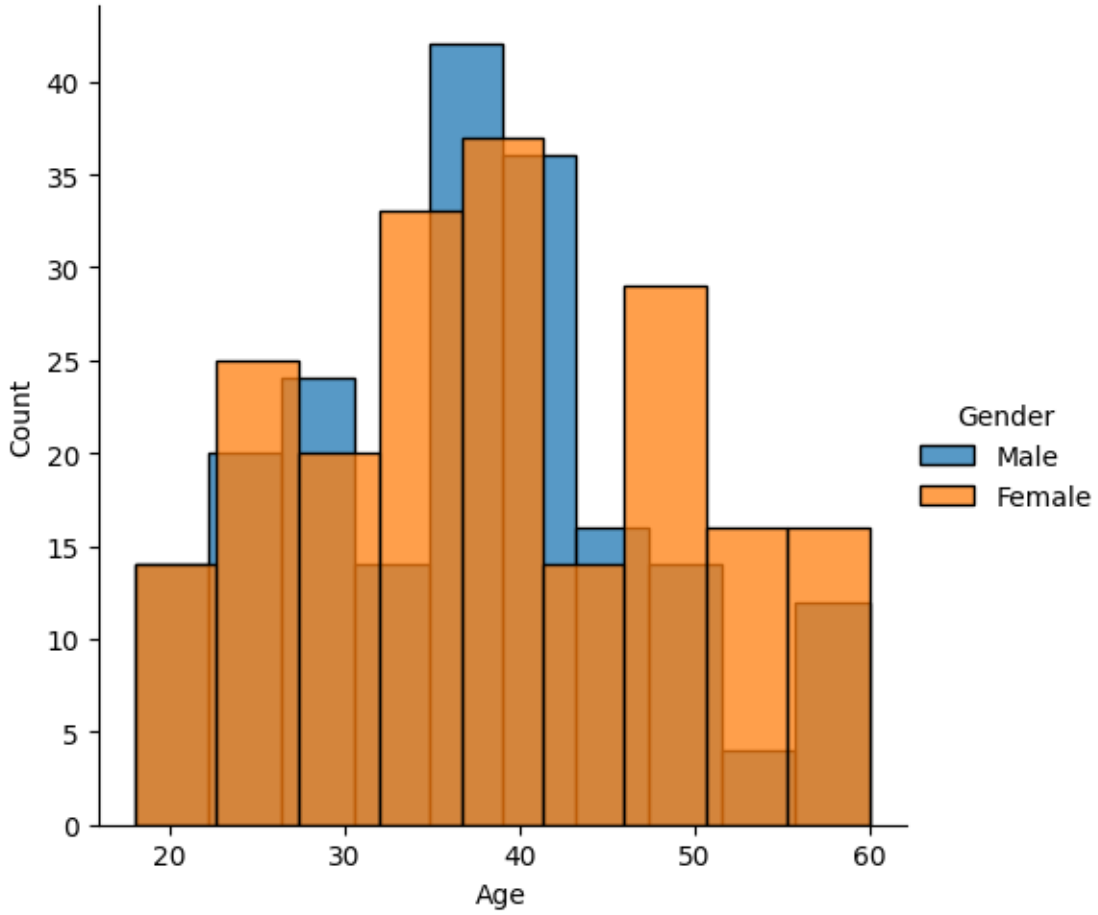

200
175
150
125
100
75
50
25
0
count
Male
Female
Gender


Gender
Male
Female
140000
120000
100000
80000
60000
40000
20000
EstimatedSalary
20
30
40
50
60
Age


Purchased
0
1
140000
120000
100000
80000
60000
40000
20000
EstimatedSalary
20
30
40
50
60
Age


Age
EstimatedSalary
Purchased
0
1
60
50
40
30
20
140000
120000
100000
80000
60000
40000
20000
20
40
60
0
50000
100000
150000
200000


Count
0
10
20
30
40
20000
40000
60000
80000
100000
120000
140000
EstimatedSalary
Gender
Male
Female



<class 'pandas.core.frame.DataFrame'>

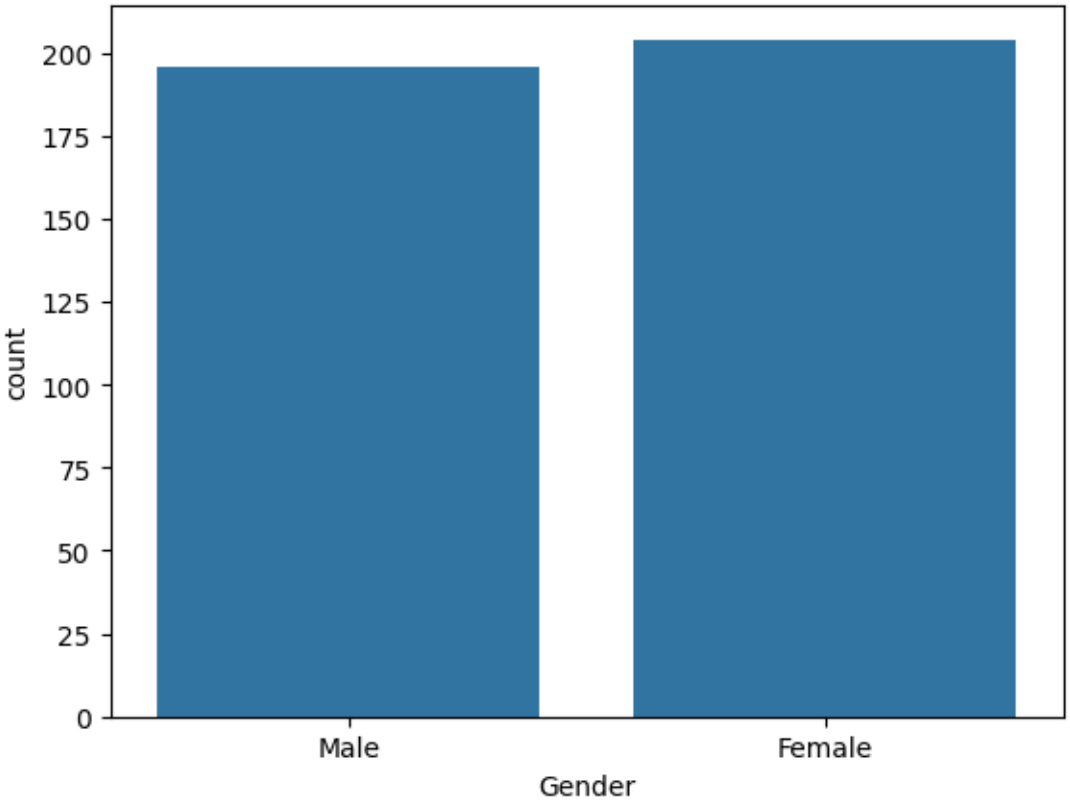
RangeIndex: 400 entries, 0 to 399

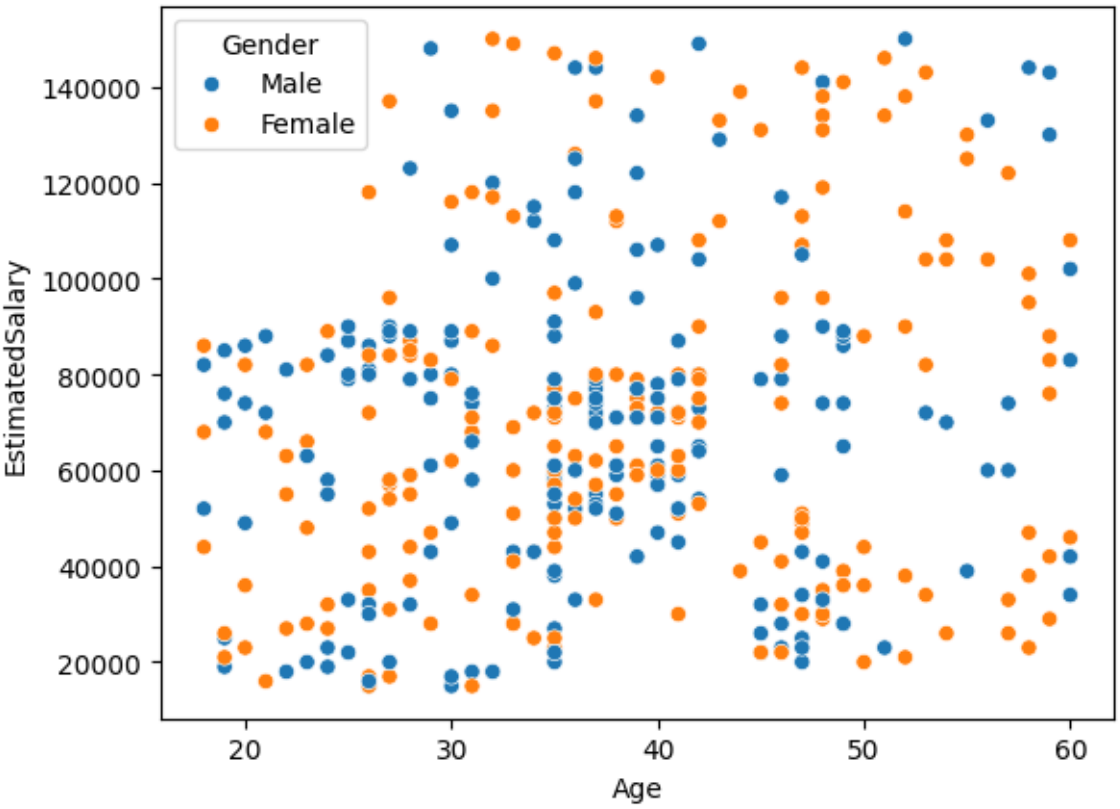
Data columns (total 5 columns):

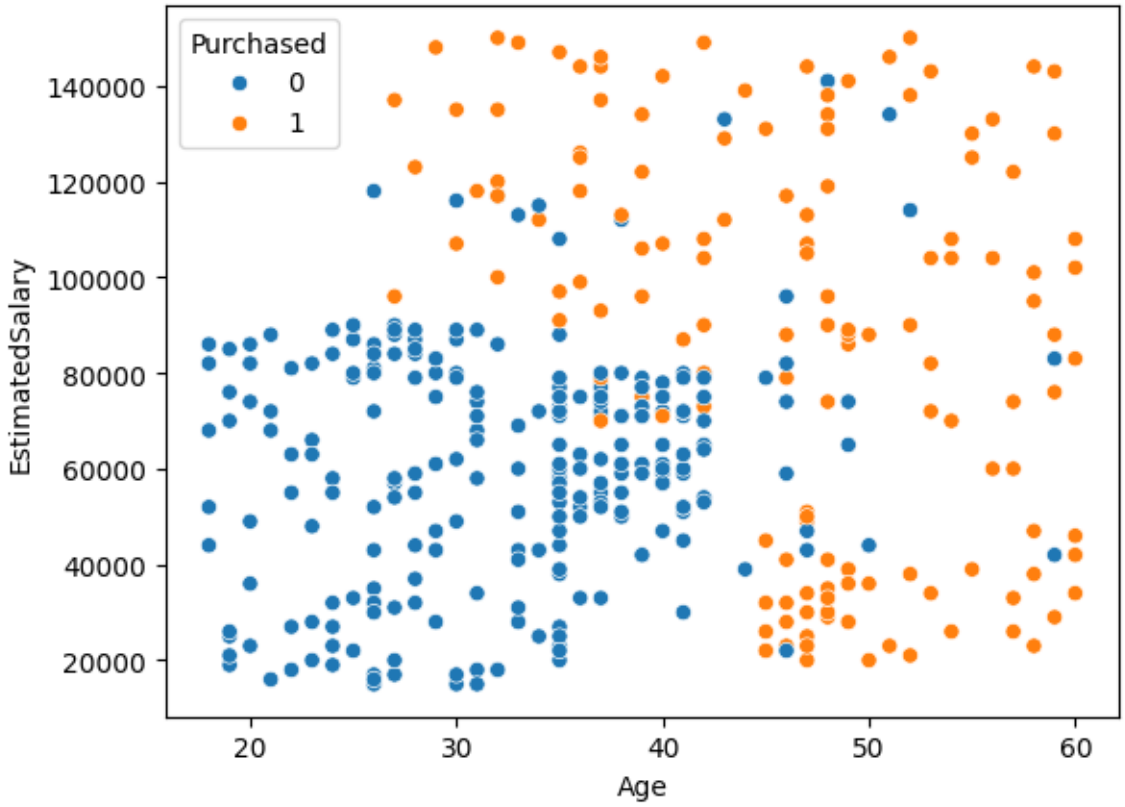
#	Column	Non-Null Count	Dtype
0	User ID	400 non-null	int64
1	Gender	400 non-null	object
2	Age	400 non-null	int64
3	EstimatedSalary	400 non-null	int64
4	Purchased	400 non-null	int64

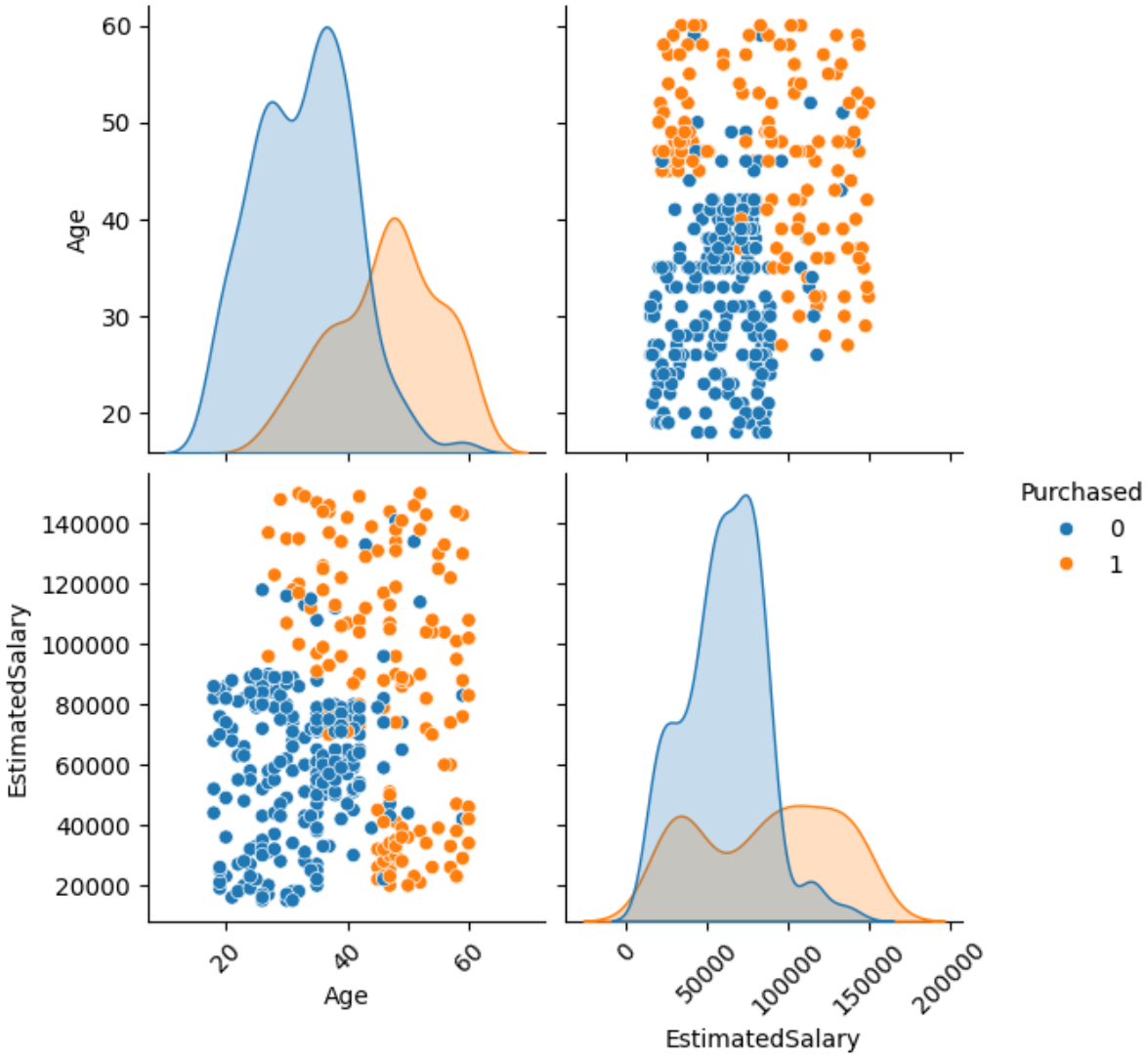
dtypes: int64(4), object(1)

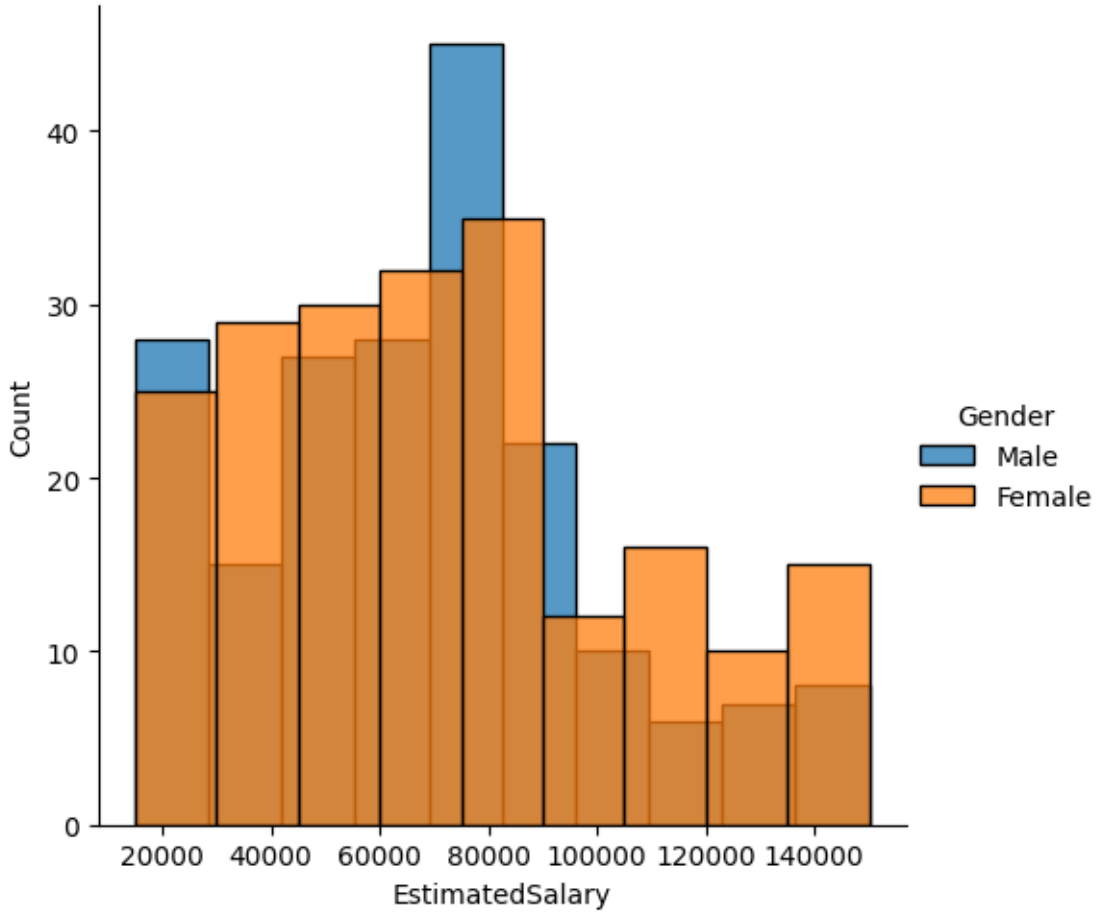
memory usage: 15.8+ KB

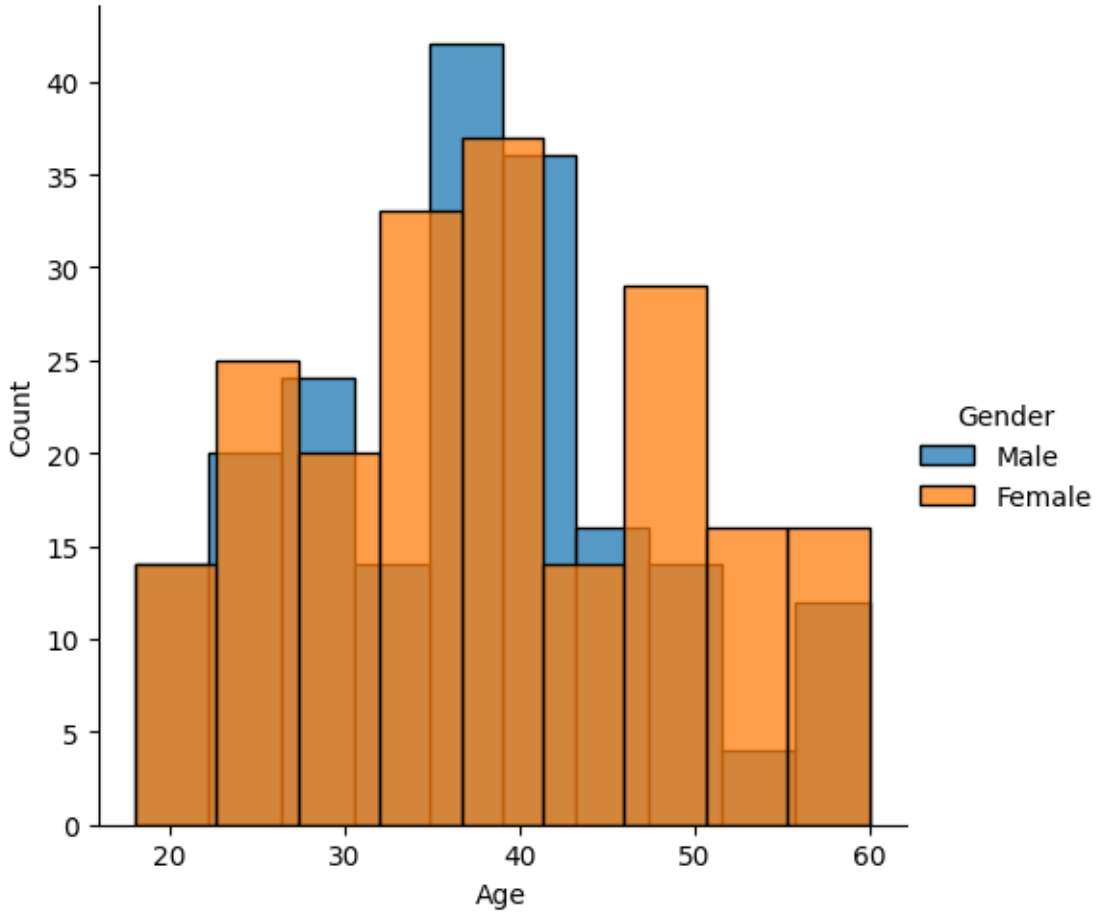

200
175
150
125
100
75
50
25
0
count
Male
Female
Gender


Gender
Male
Female
140000
120000
100000
80000
60000
40000
20000
EstimatedSalary
20
30
40
50
60
Age


Purchased
0
1
140000
120000
100000
80000
60000
40000
20000
EstimatedSalary
20
30
40
50
60
Age


Age
EstimatedSalary
Purchased
0
1
60
50
40
30
20
140000
120000
100000
80000
60000
40000
20000
20
40
60
0
50000
100000
150000
200000


Count
0
10
20
30
40
20000
40000
60000
80000
100000
120000
140000
EstimatedSalary
Gender
Male
Female



Result:

The iris dataset was successfully visualized using different plots,helping to understand the distributions and relationships among the data features.

Exp No:10 Experiment to understand array function in Data science.

AIM:

To understand array function in Data science.

Description:

Understand array function using Numpy library

CODE:

```
import numpy as np
array = np.array([15, 15, 16, 16, 17, 17, 18, 18, 19])
print("1. Original Array:")
print(array)
print("\n2. Square Root of Array:")
print(np.sqrt(array))
print("\n3. Number of Dimensions:")
print(array.ndim)
new_array = array.reshape(3, 3)
print("\n4. Reshaped Array:")
print(new_array)
print("\n5. Number of Dimensions after Reshape:")
print(new_array.ndim)
print("\n6. Ravelled Array:")
print(new_array.ravel())
newm = new_array.reshape(3, 3)
print("\n7. New Array:")
print(newm)
print("\n8. newm[2,1:3]:")
print(newm[2,1:3])
print("\n9. newm[1:2,1:3]:")
print(newm[1:2,1:3])
```

```
print("\n10. new_array[0:3,0:0]:")
print(new_array[0:3,0:0])
print("\n11. new_array[0:2,0:1]:")
print(new_array[0:2,0:1])
print("\n12. new_array[0:3,0:1]:")
print(new_array[0:3,0:1])
print("\n13. new_array[1:3]:")
print(new_array[1:3])
```

OUTPUT:

```
1. Original Array:
[15 15 16 16 17 17 18 18 19]
2. Square Root of Array:
[3.87298335 3.87298335 4.        4.        4.12310563 4.12310563
 4.24264069 4.24264069 4.35889894]
3. Number of Dimensions:
1
4. Reshaped Array:
[[15 15 16]
 [16 17 17]
 [18 18 19]]
5. Number of Dimensions after Reshape:
2
6. Ravelled Array:
[15 15 16 16 17 17 18 18 19]
7. New Array:
[[15 15 16]
 [16 17 17]
```

[18 18 19]]

8. newm[2,1:3]:

[18 19]

9. newm[1:2,1:3]:

[[17 17]]

10. new_array[0:3,0:0]:

[]

11. new_array[0:2,0:1]:

[[15]

[16]]

12. new_array[0:3,0:1]:

[[15]

[16]

[18]]

13. new_array[1:3]:

[[16 17 17]

[18 18 19]]

RESULT:

The array operations ere successfully performed using numpy including reshaping,indexing,slicing,ravelling.

Exp No:11 Experiment to understand panda library use cases in Data science.

AIM:

To understand panda library use cases in Data science.

Description:

Understand data frame use cases using panda library

CODE:

```
import numpy as np
import pandas as pd
list1 = [
   [1, "Arun", 85],
   [2, "Priya", 92]
]
df = pd.DataFrame(list1)
df
df.columns = ["ID", "Name", "Marks"]
df
df.info()
df = pd.read_csv("student.csv")
df
df.info()
df.head()
df.tail()
df["Marks"]
type(df["Marks"])
df["Marks"].mean()
df["Marks"].median()
df["Marks"].mode()
df["Marks"].var()
```

```
df.describe()

df.describe(include='all')

empcol = df.columns

empcol

emparray = df.values

emparray

student_df = pd.DataFrame(
   emparray,
   columns=empcol
)

student_df
```

OUTPUT:

	0	1	2
0	1	Arun	85
1	2	Priya	92

	Name	Age	Marks	Department
3	Divya	20	88	EEE
4	Kiran	19	95	CSE
5	Meena	20	81	ECE
6	Ajay	19	73	EEE
7	Sneha	20	90	CSE

	ID	Name	Marks
0	1	Arun	85
1	2	Priya	92

	Name	Age	Marks	Department
0	Arun	19	85	CSE
1	Priya	20	92	CSE
2	Ravi	19	76	ECE
3	Divya	20	88	EEE
4	Kiran	19	95	CSE
5	Meena	20	81	ECE
6	Ajay	19	73	EEE
7	Sneha	20	90	CSE

	Name	Age	Marks	Department
0	Arun	19	85	CSE
1	Priya	20	92	CSE
2	Ravi	19	76	ECE
3	Divya	20	88	EEE
4	Kiran	19	95	CSE

	Name	Age	Marks	Department
3	Divya	20	88	EEE
4	Kiran	19	95	CSE
5	Meena	20	81	ECE
6	Ajay	19	73	EEE
7	Sneha	20	90	CSE

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 8 entries, 0 to 7

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	Name	8 non-null	object
1	Age	8 non-null	int64
2	Marks	8 non-null	int64
3	Department	8 non-null	object

dtypes: int64(2), object(2)

memory usage: 388.0+ bytes

np.float64(85.0)

86.5

	Marks
0	73
1	76
2	81
3	85
4	88
5	90
6	92
7	95

dtype: int64

60.57142857142857

7.782764841072135

	Age	Marks
count	8.000000	8.000000
mean	19.500000	85.000000
std	0.534522	7.782765
min	19.000000	73.000000
25%	19.000000	79.750000
50%	19.500000	86.500000
75%	20.000000	90.500000
max	20.000000	95.000000

	Name	Age	Marks	Department
count	8	8.000000	8.000000	8
unique	8	NaN	NaN	3
top	Arun	NaN	NaN	CSE
freq	1	NaN	NaN	4
mean	NaN	19.500000	85.000000	NaN
std	NaN	0.534522	7.782765	NaN
min	NaN	19.000000	73.000000	NaN
25%	NaN	19.000000	79.750000	NaN
50%	NaN	19.500000	86.500000	NaN
75%	NaN	20.000000	90.500000	NaN
max	NaN	20.000000	95.000000	NaN

Index(['Name', 'Age', 'Marks', 'Department'], dtype='object')

	Name	Age	Marks	Department
0	Arun	19	85	CSE
1	Priya	20	92	CSE
2	Ravi	19	76	ECE
3	Divya	20	88	EEE
4	Kiran	19	95	CSE
5	Meena	20	81	ECE
6	Ajay	19	73	EEE
7	Sneha	20	90	CSE

RESULT:

The pandas library was successfully used to create load and explore a data frame.various dataframe operations and statistical functions were performed successfully of the student datset.

Exp No:12 Experiment to detect outliers in a given data set.

AIM:

To detect outliers in a given data set.

Description:

Understand the procedure to identify the outliers in a given dataset.

CODE:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv("student_marks_outliers.csv")
df
array = df["Marks"].values
array
array.mean()
print("25th Percentile:", np.percentile(array, 25))
print("50th Percentile:", np.percentile(array, 50))
print("75th Percentile:", np.percentile(array, 75))
print("100th Percentile:", np.percentile(array, 100))
Q1 = np.percentile(array, 25)
Q3 = np.percentile(array, 75)
IQR = Q3 - Q1
print("Q1 =", Q1)
print("Q3 =", Q3)
print("IQR =", IQR)
lower_limit = Q1 - (1.5 * IQR)
upper_limit = Q3 + (1.5 * IQR)
print("Lower Limit:", lower_limit)
```

```
print("Upper Limit:", upper_limit)
outliers = array[(array < lower_limit) | (array > upper_limit)]
print("Outliers:")
print(outliers)
def outDetection(array):
   Q1 = np.percentile(array, 25)
   Q3 = np.percentile(array, 75)
   IQR = Q3 - Q1
   lr = Q1 - (1.5 * IQR)
   ur = Q3 + (1.5 * IQR)
   return lr, ur
 lr, ur = outDetection(array)
print("Lower Range:", lr)
print("Upper Range:", ur)
sns.histplot(array, bins=8, kde=True)
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Count")
plt.show()
```

output:

25th Percentile: 72.0

50th Percentile: 79.0

75th Percentile: 87.0

100th Percentile: 130.0

Q1 = 72.0

Q3 = 87.0

IQR = 15.0

Lower Limit: 49.5

Upper Limit: 109.5

Outliers:

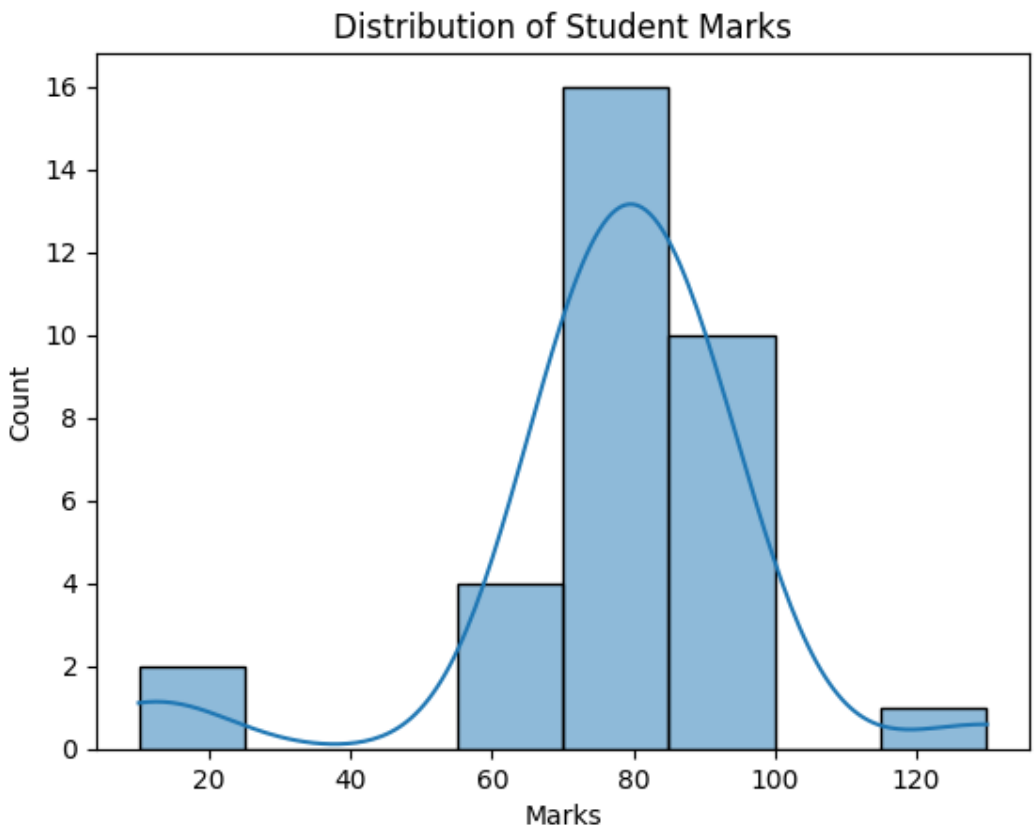
[10 15 130]

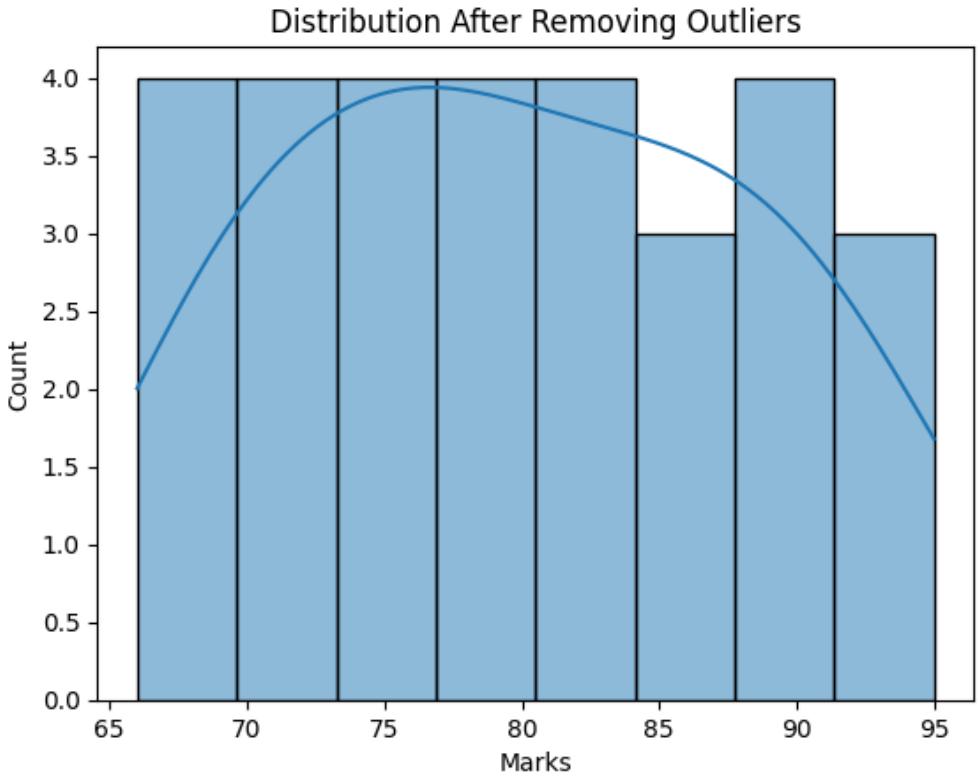
Lower Range: 49.5

Upper Range: 109.5

array([72, 68, 75, 81, 77, 69, 85, 74, 80, 78, 83, 71, 76, 88, 79, 82, 73,

86, 90, 84, 67, 70, 92, 87, 89, 94, 66, 91, 95, 76])


Distribution of Student Marks
Count
Marks
0
2
4
6
8
10
12
14
16
20
40
60
80
100
120


Distribution After Removing Outliers
Count
Marks
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
65
70
75
80
85
90
95

Result:

Outliers in the student marks dataset were successfully identified using the IQR method and the effect of removing them was visualized.

Exp No:13 Experiment to handle missing values and inappropriate data in a given data set.

AIM:

To handle missing values and inappropriate data in a given data set.

Description:

Understand the procedure to handle missing and inappropriate data.

CODE:

```
import numpy as np
import pandas as pd
df = pd.read_csv("student_missing_inappropriate.csv")
df
df.duplicated()
df.info()
df.drop_duplicates(inplace=True)
df
len(df)
index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
index
df
df.loc[df["Marks"] < 0, "Marks"] = np.nan
df
df.loc[df["Attendance"] < 0, "Attendance"] = np.nan
df
df.loc[df["Fees"] < 0, "Fees"] = np.nan
df
df["Department"].unique()
```

```
df["Marks"].fillna(
   round(df["Marks"].median()),
   inplace=True
)
df
df["Attendance"].fillna(
   round(df["Attendance"].median()),
   inplace=True
)
df
df["Fees"].fillna(
   round(df["Fees"].mean()),
   inplace=True
)
Df
```

OUTPUT:

	0
0	False
1	False
2	False
3	False
4	False
5	True
6	False
7	False
8	False

	0
9	False

dtype: bool

	StudentID	Name	Age	Marks	Department	Attendance	Fees
0	1	Arun	19	85	CSE	92	50000
1	2	Priya	20	92	CSE	88	52000
2	3	Ravi	19	76	ECE	75	48000
3	4	Divya	20	-5	EEE	91	51000
4	5	Kiran	21	88	CSE	86	53000
6	7	Meena	20	81	ECE	80	-1000
7	8	Ajay	19	95	EEE	95	50000
8	9	Sneha	20	-10	CSE	89	54000
9	10	Vijay	22	90	CSE	-1	55000

	StudentID	Name	Age	Marks	Department	Attendance	Fees
0	1	Arun	19	85.0	CSE	92.0	50000.0
1	2	Priya	20	92.0	CSE	88.0	52000.0
2	3	Ravi	19	76.0	ECE	75.0	48000.0
3	4	Divya	20	NaN	EEE	91.0	51000.0
4	5	Kiran	21	88.0	CSE	86.0	53000.0
5	7	Meena	20	81.0	ECE	80.0	NaN
6	8	Ajay	19	95.0	EEE	95.0	50000.0

	StudentID	Name	Age	Marks	Department	Attendance	Fees
7	9	Sneha	20	NaN	CSE	89.0	54000.0
8	10	Vijay	22	90.0	CSE	NaN	55000.0

	StudentID	Name	Age	Marks	Department	Attendance	Fees
0	1	Arun	19	85.0	CSE	92.0	50000.0
1	2	Priya	20	92.0	CSE	88.0	52000.0
2	3	Ravi	19	76.0	ECE	75.0	48000.0
3	4	Divya	20	88.0	EEE	91.0	51000.0
4	5	Kiran	21	88.0	CSE	86.0	53000.0
5	7	Meena	20	81.0	ECE	80.0	51625.0
6	8	Ajay	19	95.0	EEE	95.0	50000.0
7	9	Sneha	20	88.0	CSE	89.0	54000.0
8	10	Vijay	22	90.0	CSE	88.0	55000.0

RESULT:

The duplicate and inappropriate values are successfully identified and handled and the dataset was cleaned using pandas

Exp No:14 Experiment to understand feature scaling.

AIM:

To understand and implement feature scaling techniques on a given dataset.

Description:

Understand the importance of feature scaling.

CODE:

```
import numpy as np
import pandas as pd
from google.colab import files
uploaded = files.upload()
df = pd.read_csv("feature_scaling_dataset.csv")
df
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
df
features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values
features
from sklearn.impute import SimpleImputer
age_imputer = SimpleImputer(strategy="mean")
salary_imputer = SimpleImputer(strategy="mean")
age_imputer.fit(features[:, [1]])
salary_imputer.fit(features[:, [2]])
features[:, [1]] = age_imputer.transform(features[:, [1]])
features[:, [2]] = salary_imputer.transform(features[:, [2]])
features
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])
```

```
Country
final_set = np.concatenate(
   (Country, features[:, [1, 2]]),
   axis=1
)
final_set
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
sc.fit(final_set)
feat_standard_scaler = sc.transform(final_set)
feat_standard_scaler
from sklearn.preprocessing import MinMaxScaler
mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)
feat_minmax_scaler = mms.transform(final_set)
feat_minmax_scaler
```

OUTPUT:

1. Original Dataset

	Country	Age	Salary	Purchased
0	France	42.0	68000.0	No
1	Spain	29.0	51000.0	Yes
2	Germany	31.0	57000.0	No
3	Spain	36.0	64000.0	Yes
4	Germany	NaN	NaN	No
5	France	34.0	61000.0	Yes
6	Spain	NaN	55000.0	No
7	France	46.0	82000.0	Yes
8	Germany	51.0	76000.0	No
9	France	39.0	70000.0	Yes

2. Features After Missing Values Are Filled

```
array([['France', 42.0, 68000.0],
       ['Spain', 29.0, 51000.0],
       ['Germany', 31.0, 57000.0],
       ['Spain', 36.0, 64000.0],
       ['Germany', 38.5, 64888.88888888889],
       ['France', 34.0, 61000.0],
       ['Spain', 38.5, 55000.0],
       ['France', 46.0, 82000.0],
       ['Germany', 51.0, 76000.0],
       ['France', 39.0, 70000.0]], dtype=object)
```

3. One-Hot Encoding

```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

4. Final Dataset Before Scaling

```
array([[1.0, 0.0, 0.0, 42.0, 68000.0],
       [0.0, 0.0, 1.0, 29.0, 51000.0],
       [0.0, 1.0, 0.0, 31.0, 57000.0],
       [0.0, 0.0, 1.0, 36.0, 64000.0],
       [0.0, 1.0, 0.0, 38.5, 64888.88888888889],
       [1.0, 0.0, 0.0, 34.0, 61000.0],
       [0.0, 0.0, 1.0, 38.5, 55000.0],
```

[1.0, 0.0, 0.0, 46.0, 82000.0],
[0.0, 1.0, 0.0, 51.0, 76000.0],
[1.0, 0.0, 0.0, 39.0, 70000.0]], dtype=object)

5. StandardScaler Output

```
array([[ 1.22474487, -0.65465367, -0.65465367,  0.5547873 ,  0.34337869],
       [-0.81649658, -0.65465367,  1.52752523, -1.50585123, -1.53294056],
       [-0.81649658,  1.52752523, -0.65465367, -1.18882992, -0.87071024],
       [-0.81649658, -0.65465367,  1.52752523, -0.39627664, -0.0981082 ],
       [-0.81649658,  1.52752523, -0.65465367,  0.        ,  0.        ],
       [ 1.22474487, -0.65465367, -0.65465367, -0.71329795, -0.42922336],
       [-0.81649658, -0.65465367,  1.52752523,  0.        , -1.09145368],
       [ 1.22474487, -0.65465367, -0.65465367,  1.18882992,  1.88858277],
       [-0.81649658,  1.52752523, -0.65465367,  1.9813832 ,  1.22635245],
       [ 1.22474487, -0.65465367, -0.65465367,  0.07925533,  0.56412213]])
```

6. MinMaxScaler Output

```
array([[1.        , 0.        , 0.        , 0.59090909, 0.5483871 ],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.09090909, 0.19354839],
       [0.        , 0.        , 1.        , 0.31818182, 0.41935484],
       [0.        , 1.        , 0.        , 0.43181818, 0.44802867],
       [1.        , 0.        , 0.        , 0.22727273, 0.32258065],
       [0.        , 0.        , 1.        , 0.43181818, 0.12903226],
       [1.        , 0.        , 0.        , 0.77272727, 1.        ],
       [0.        , 1.        , 0.        , 1.        , 0.80645161],
       [1.        , 0.        , 0.        , 0.45454545, 0.61290323]])
```

RESULT:

The dataset was successfully preprocessed and scaled using standard scaler and minmax.

Exp No:15 Experiment to understand the data preprocessing in Data Science

AIM:

To understand the data preprocessing in Data Science

Description:

Understand the importance of Data preprocessing in data science.

CODE:

```
import numpy as np
import pandas as pd
df = pd.read_csv("/content/customer_purchase_dataset.csv")
df
df.info()
df.City.mode()
df.City.mode()[0]
type(df.City.mode())
df.City.fillna(df.City.mode()[0], inplace=True)
df.Age.fillna(df.Age.median(), inplace=True)
df.Income.fillna(round(df.Income.mean()), inplace=True)
df
pd.get_dummies(df.City)
updated_dataset = pd.concat(
  [pd.get_dummies(df.City), df.iloc[:, [1, 2, 4, 5]]],
  axis=1
)
updated_dataset
updated_dataset.info()
updated:_dataset.Purchased.replace(['No', 'Yes'], [0, 1], inplace=True)
```

updated_dataset

OUTPUT:

	Customer_ID	Gender	Age	City	Income	Purchased
0	301	Male	22.0	Chennai	25000.0	No
1	302	Female	25.0	Bangalore	40000.0	Yes
2	303	Male	28.0	Chennai	52000.0	Yes
3	304	Female	24.0	Hyderabad	NaN	No
4	305	Male	30.0	Bangalore	65000.0	Yes
5	306	Female	27.0	Chennai	48000.0	Yes
6	307	Male	NaN	Hyderabad	35000.0	No
7	308	Female	29.0	Bangalore	NaN	Yes
8	309	Male	32.0	Chennai	72000.0	Yes
9	310	Female	26.0	Hyderabad	42000.0	No

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10 entries, 0 to 9

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Customer_ID	10 non-null	int64
1	Gender	10 non-null	object
2	Age	9 non-null	float64
3	City	10 non-null	object
4	Income	8 non-null	float64
5	Purchased	10 non-null	object

dtypes: float64(2), int64(1), object(3)

memory usage: 612.0+ bytes

	City
0	Chennai

dtype: object

	Customer_ID	Gender	Age	City	Income	Purchased
0	301	Male	22.0	Chennai	25000.0	No
1	302	Female	25.0	Bangalore	40000.0	Yes
2	303	Male	28.0	Chennai	52000.0	Yes
3	304	Female	24.0	Hyderabad	47375.0	No
4	305	Male	30.0	Bangalore	65000.0	Yes
5	306	Female	27.0	Chennai	48000.0	Yes
6	307	Male	27.0	Hyderabad	35000.0	No
7	308	Female	29.0	Bangalore	47375.0	Yes
8	309	Male	32.0	Chennai	72000.0	Yes
9	310	Female	26.0	Hyderabad	42000.0	No

	Bangalore	Chennai	Hyderabad
0	False	True	False
1	True	False	False
2	False	True	False
3	False	False	True
4	True	False	False
5	False	True	False

	Bangalore	Chennai	Hyderabad
6	False	False	True
7	True	False	False
8	False	True	False
9	False	False	True

	Bangalore	Chennai	Hyderabad	Gender	Age	Income	Purchased
0	False	True	False	Male	22.0	25000.0	No
1	True	False	False	Female	25.0	40000.0	Yes
2	False	True	False	Male	28.0	52000.0	Yes
3	False	False	True	Female	24.0	47375.0	No
4	True	False	False	Male	30.0	65000.0	Yes
5	False	True	False	Female	27.0	48000.0	Yes
6	False	False	True	Male	27.0	35000.0	No
7	True	False	False	Female	29.0	47375.0	Yes
8	False	True	False	Male	32.0	72000.0	Yes
9	False	False	True	Female	26.0	42000.0	No

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10 entries, 0 to 9

Data columns (total 7 columns):

Column Non-Null Count Dtype

--- ------ -------------- -----

0 Bangalore 10 non-null bool

1 Chennai 10 non-null bool

2 Hyderabad 10 non-null bool

3 Gender 10 non-null object

4 Age 10 non-null float64

5 Income 10 non-null float64

6 Purchased 10 non-null object

dtypes: bool(3), float64(2), object(2)

memory usage: 482.0+ bytes

	Bangalore	Chennai	Hyderabad	Gender	Age	Income	Purchased
0	False	True	False	Male	22.0	25000.0	0
1	True	False	False	Female	25.0	40000.0	1
2	False	True	False	Male	28.0	52000.0	1
3	False	False	True	Female	24.0	47375.0	0
4	True	False	False	Male	30.0	65000.0	1
5	False	True	False	Female	27.0	48000.0	1
6	False	False	True	Male	27.0	35000.0	0
7	True	False	False	Female	29.0	47375.0	1
8	False	True	False	Male	32.0	72000.0	1
9	False	False	True	Female	26.0	42000.0	0

RESULT:

The given dataset was successfully preprocessed by handling missing values and preparing it for further analaysis.

Exp No:16 Experiment to understand the data preprocessing in Data Science

AIM:

To understand the data preprocessing in Data Science

Description:

Understand the importance of Data preprocessing in data science

CODE:

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
df = pd.read_csv("/content/student_eda_dataset.csv")
df
df.head()
sns.displot(df.Marks, kde=True)
sns.displot(df.Marks, kde=False)
sns.jointplot(x=df.Study_Hours, y=df.Marks)
sns.jointplot(x=df.Study_Hours, y=df.Marks, kind="reg")
sns.jointplot(x=df.Study_Hours, y=df.Marks, kind="hex")
sns.pairplot(df[['Study_Hours', 'Attendance', 'Marks', 'Assignment_Score']])
sns.pairplot(
  df[['Study_Hours', 'Attendance', 'Marks', 'Assignment_Score', 'Study_Mode']],
  hue='Study_Mode'
)
sns.pairplot(
  df[['Study_Hours', 'Attendance', 'Marks', 'Assignment_Score', 'Gender']],
```

```
 hue='Gender'
)
sns.heatmap(
   df[['Study_Hours', 'Attendance', 'Marks', 'Assignment_Score']].corr(),
   annot=True
)
sns.boxplot(y=df.Marks)
sns.boxplot(y=df.Study_Hours)
sns.countplot(x=df.Department)
sns.countplot(x=df.Gender)
df.Gender.value_counts().plot(kind='pie', autopct='%1.1f%%')
plt.ylabel('')
plt.show()
df.Gender.value_counts().plot(kind='bar')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.show()
sns.countplot(x=df.Passed)
sns.countplot(x=df.Study_Mode)
sns.countplot(x=df.Gender, hue=df.Passed)
sns.countplot(x=df[df.Study_Mode == 'Online']['Department'])
```

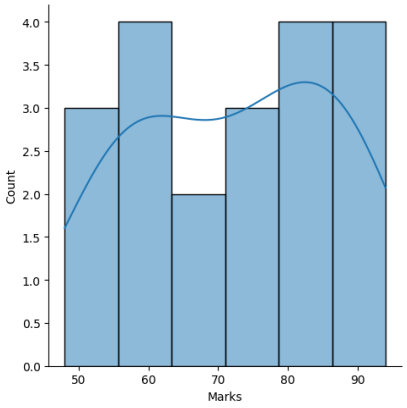
OUTPUT:

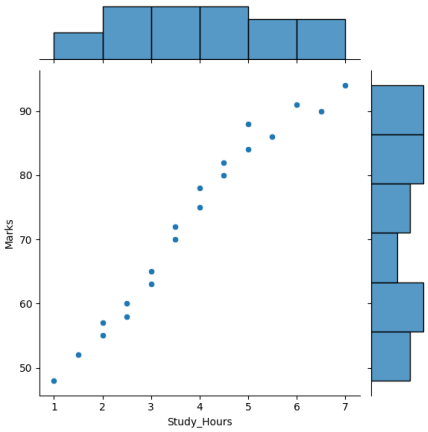
	Student_ID	Gender	Department	Study_Mode	Study_Hours	Attendance	Marks	Assignment_Score	Passed
0	101	Male	CSE	Online	2.5	72	58	60	No

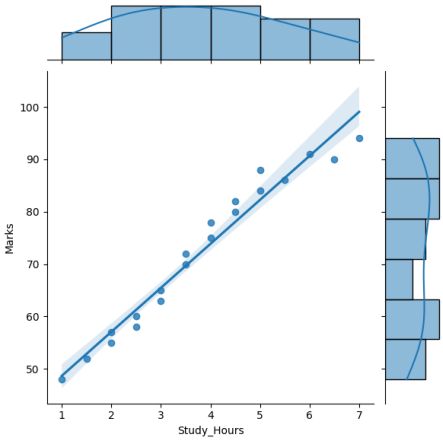
	Student_ID	Gender	Department	Study_Mode	Study_Hours	Attendance	Marks	Assignment_Score	Passed
1	102	Female	ECE	Offline	4.0	85	78	80	Yes
2	103	Male	CSE	Online	3.0	78	65	68	Yes
3	104	Female	IT	Offline	5.0	92	88	90	Yes
4	105	Male	ECE	Online	1.5	68	52	55	No
5	106	Female	CSE	Offline	6.0	95	91	93	Yes
6	107	Male	IT	Online	3.5	80	70	72	Yes
7	108	Female	ECE	Offline	4.5	88	82	85	Yes
8	109	Male	CSE	Online	2.0	70	55	58	No
9	110	Female	IT	Offline	5.5	90	86	88	Yes
10	111	Male	ECE	Offline	7.0	96	94	96	Yes
11	112	Female	CSE	Online	3.0	76	63	65	Yes
12	113	Male	IT	Offline	4.0	84	75	78	Yes
13	114	Female	ECE	Online	2.5	73	60	62	Yes
14	115	Male	CSE	Offline	6.5	94	90	92	Yes
15	116	Female	IT	Online	1.0	65	48	50	No

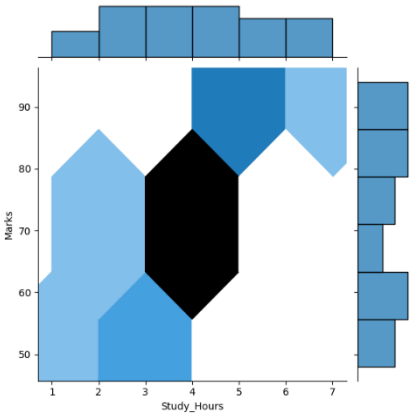
	Student_ID	Gender	Department	Study_Mode	Study_Hours	Attendance	Marks	Assignment_Score	Passed
16	117	Male	ECE	Offline	5.0	89	84	87	Yes
17	118	Female	CSE	Online	3.5	82	72	75	Yes
18	119	Male	IT	Offline	2.0	71	57	59	No
19	120	Female	ECE	Online	4.5	87	80	83	Yes

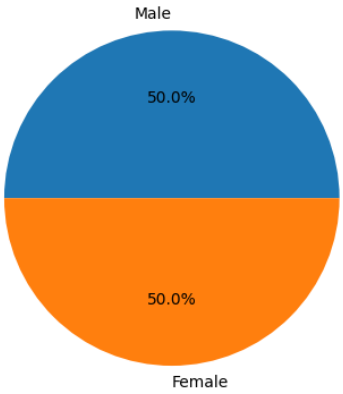
	Student_ID	Gender	Department	Study_Mode	Study_Hours	Attendance	Marks	Assignment_Score	Passed
0	101	Male	CSE	Online	2.5	72	58	60	No
1	102	Female	ECE	Offline	4.0	85	78	80	Yes
2	103	Male	CSE	Online	3.0	78	65	68	Yes
3	104	Female	IT	Offline	5.0	92	88	90	Yes
4	105	Male	ECE	Online	1.5	68	52	55	No

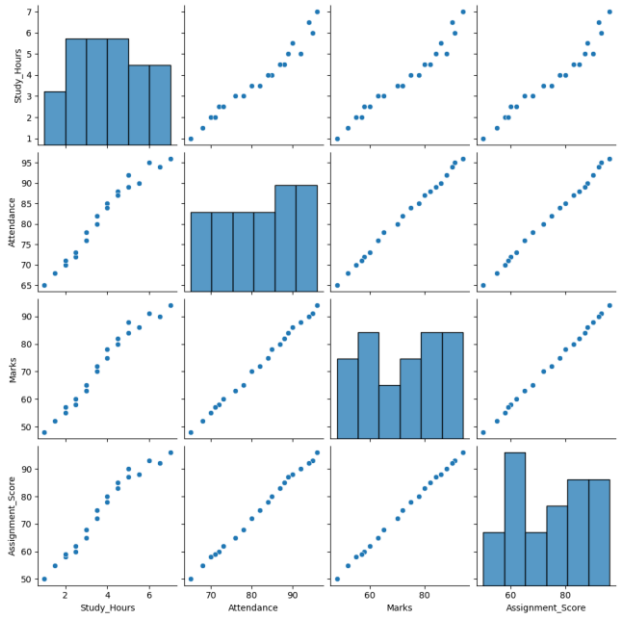


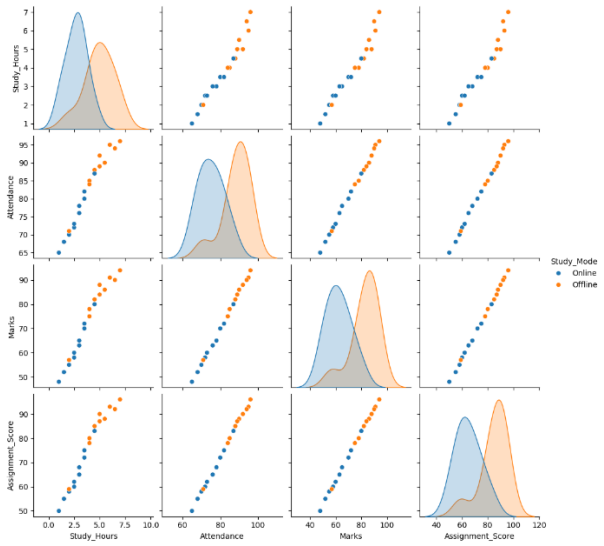


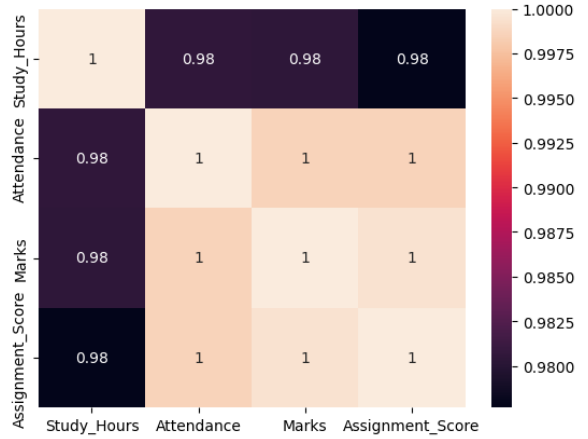

Marks
Study_Hours
100
90
80
70
60
50
1
2
3
4
5
6
7

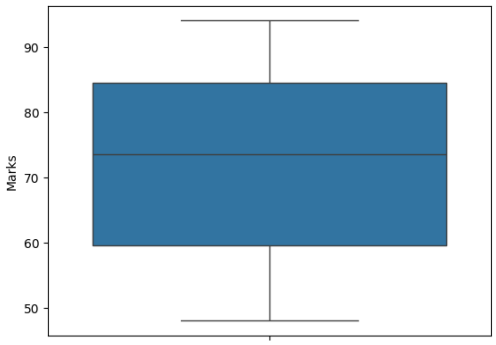

Marks
Study_Hours
90
80
70
60
50
1
2
3
4
5
6
7

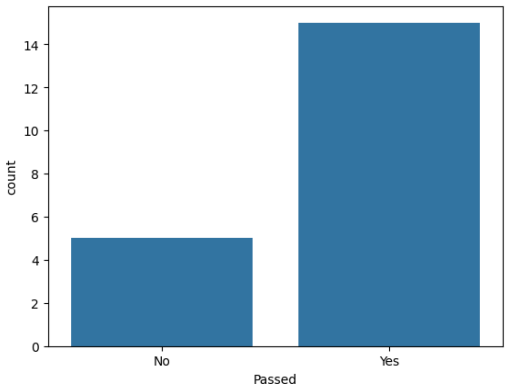

Male
50.0%
50.0%
Female

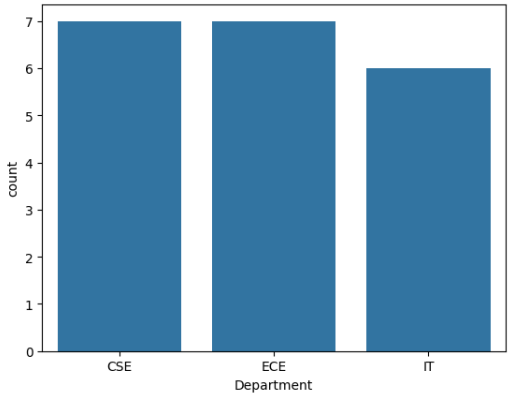

Study_Hours
Attendance
Marks
Assignment_Score

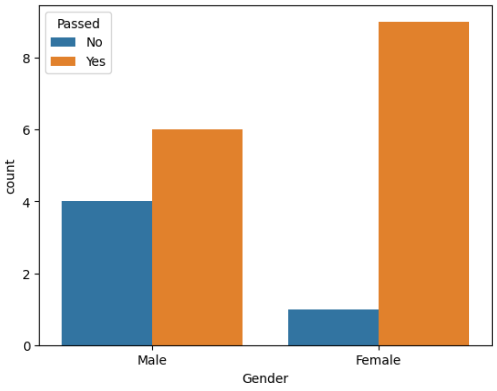

Study_Mode
Online
Offline
Study_Hours
Attendance
Marks
Assignment_Score

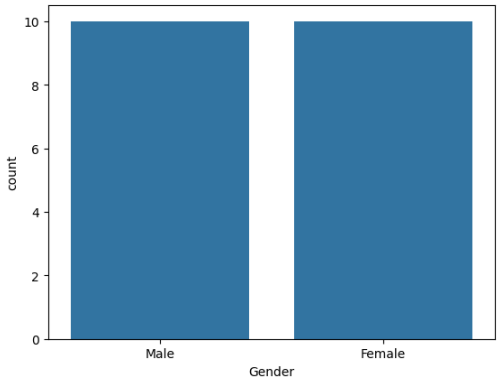

Study_Hours
Attendance
Marks
Assignment_Score
1
0.98
0.98
0.98
0.98
1
1
1
0.98
1
1
1
0.98
1
1
1
1.0000
0.9975
0.9950
0.9925
0.9900
0.9875
0.9850
0.9825
0.9800

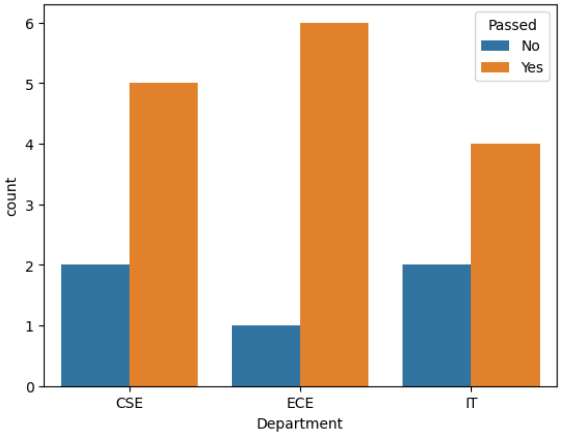












RESULT:

The given dataset is visualized by using python code.

Exp No:17 Experiment to understand Linear Regression for a given

data set.

AIM:

To implement a linear regression model for predicting marks on study hours.

Description:

Understand the Linear regression for the dataset given.

CODE:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
df
df.info()
df.dropna(inplace=True)
df.info()
df.describe()
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_st
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
model.score(x_train,y_train)
model.score(x_test,y_test)
model.coef_
model.intercept_
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
```

```
model=pickle.load(open('SalaryPred.model','rb'))

yr_of_exp=float(input("Enter Years of Experience: "))

yr_of_exp_NP=np.array([[yr_of_exp]])

Salary=model.predict(yr_of_exp_NP)

print("Estimated Salary for {} years of experience is {}: " .format(yr_of_exp,Salary)
```

OUTPUT:

1. Original Dataset

```
   StudyHours  Marks
0          1    35
1          2    40
2          3    45
3          4    50
4          5    55
5          6    60
6          7    65
7          8    70
8          9    74
9         10    79
10        11    83
11        12    87
12        13    91
13        14    94
14        15    97
```

2. Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   StudyHours  15 non-null     int64
 1   Marks       15 non-null     int64
dtypes: int64(2)
memory usage: 372.0 bytes
```

3. Statistical Summary

```
       StudyHours     Marks
count   15.000000  15.000000
```

mean	8.000000	68.333333
std	4.472136	20.282528
min	1.000000	35.000000
25%	4.500000	52.500000
50%	8.000000	70.000000
75%	11.500000	85.000000
max	15.000000	97.000000

4. Linear Regression Model

LinearRegression()

5. Training Score

0.995281820510348

6. Testing Score

0.9961244386254701

7. Model Coefficient

array([[4.51970701]])

8. Model Intercept

array([32.0523195])

9. Final Prediction

Enter Study Hours: 4
Estimated Marks for 4.0 hours of study is [[50.13114754]]

RESULT:

The linear regression model was successfully created and used to predict marks based on the number of study hours.

Exp No:18 Experiment to understand KNN algorithm for a given data

Set

AIM:

To implement the k-nearest neighbours classification algorithm to satisfy fruits based on their weight and sweetness.

Description:

Understand the KNN algorithm for the dataset given.

CODE:

```
import numpy as np

import pandas as pd

df=pd.read_csv('Iris.csv')

df.info()

df.variety.value_counts()

df.head()

features=df.iloc[:,:-1].values

label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,rando

model_KNN=KNeighborsClassifier(n_neighbors=5)

model_KNN.fit(xtrain,ytrain)

print(model_KNN.score(xtrain,ytrain))

print(model_KNN.score(xtest,ytest))

from sklearn.metrics import confusion_matrix

confusion_matrix(label,model_KNN.predict(features))

from sklearn.metrics import classification_report

print(classification_report(label,model_KNN.predict(features)))
```

OUTPUT:

1. Dataset Information

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Weight	30 non-null	int64
1	Sweetness	30 non-null	float64
2	Fruit	30 non-null	object

dtypes: float64(1), int64(1), object(1)
memory usage: 852.0+ bytes

2. Fruit Class Distribution

Fruit
Apple 10
Banana 10
Orange 10
Name: count, dtype: int64

3. Original Dataset – First Five Records

	Weight	Sweetness	Fruit
0	120	6.0	Apple
1	125	6.2	Apple
2	130	6.4	Apple
3	135	6.5	Apple
4	140	6.7	Apple

4. Label Encoding

	Weight	Sweetness	Fruit
0	120	6.0	0
1	125	6.2	0
2	130	6.4	0
3	135	6.5	0
4	140	6.7	0

5. KNN Model

KNeighborsClassifier()

6. Training and Testing Accuracy

Training Accuracy: 0.9166666666666666
Testing Accuracy: 0.6666666666666666

7. Confusion Matrix

```
array([[ 9,  0,  1],
       [ 3,  7,  0],
       [ 0,  0, 10]])
```

8. Classification Report

	precision	recall	f1-score	support
0	0.75	0.90	0.82	10
1	1.00	0.70	0.82	10
2	0.91	1.00	0.95	10
accuracy			0.87	30
macro avg	0.89	0.87	0.86	30
weighted avg	0.89	0.87	0.86	30

RESULT:

The KNN classification algorithm was successfully implemented and the fruits were classified based on their features. The model performance was evaluated using accuracy confusion matrix and classification repeat.

Exp No:19Experiment to understand Logistic Regression for a given

data set.

AIM:

Logistic regression is a supervised machine learning algorithm used to binary classification it predicts the probability of a data point belonging to a particular class.

Description:

Understand the Logistic Regression algorithm for the dataset given.

CODE:

```
import numpy as np

import pandas as pd

df=pd.read_csv('Social_Network_Ads.csv')

df

df.head()

features=df.iloc[:,[2,3]].values

label=df.iloc[:,4].values

features

label

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

for i in range(1,401):

x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.

model=LogisticRegression()

model.fit(x_train,y_train)

train_score=model.score(x_train,y_train)

test_score=model.score(x_test,y_test)

if test_score>train_score:

print("Test {} Train{} Random State {}".format(test_score,train_score,i)

x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,
```

```
finalModel=LogisticRegression()

finalModel.fit(x_train,y_train)

print(finalModel.score(x_train,y_train))

print(finalModel.score(x_test,y_test))

from sklearn.metrics import classification_report

print(classification_report(label,finalModel.predict(features)))
```

OUTPUT:

1. Original Dataset

	StudyHours	Attendance	Pass
0	1	50	0
1	2	55	0
2	2	60	0
3	3	58	0
4	3	62	0
5	3	65	0
6	4	60	0
7	4	68	1
8	4	70	1
9	5	65	0
10	5	70	1
11	5	72	1
12	6	68	1
13	6	75	1
14	6	78	1
15	7	72	1
16	7	80	1
17	7	82	1
18	8	78	1
19	8	84	1
20	8	86	1
21	9	82	1
22	9	88	1
23	9	90	1
24	10	86	1
25	10	92	1
26	10	94	1

27	11	90	1
28	11	95	1
29	12	96	1

6. Classification Report

	precision	recall	f1-score	support
0	1.00	1.00	1.00	8
1	1.00	1.00	1.00	22
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

2. Features

```
array([[ 1, 50],
       [ 2, 55],
       [ 2, 60],
       [ 3, 58],
       [ 3, 62],
       [ 3, 65],
       [ 4, 60],
       [ 4, 68],
       [ 4, 70],
       [ 5, 65],
       [ 5, 70],
       [ 5, 72],
       [ 6, 68],
       [ 6, 75],
       [ 6, 78],
       [ 7, 72],
       [ 7, 80],
       [ 7, 82],
       [ 8, 78],
       [ 8, 84],
```

```
[ 8, 86],
[ 9, 82],
[ 9, 88],
[ 9, 90],
[10, 86],
[10, 92],
[10, 94],
[11, 90],
[11, 95],
[12, 96]])
```

3. Labels

```
array([0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

4. Final Logistic Regression Model

```
LogisticRegression(max_iter=1000)
```

5. Training and Testing Accuracy

```
Training Accuracy: 1.0
Testing Accuracy: 1.0
```

RESULT:

Thus the logistic regression algorithm was successfully implemented and test to classify the given dataset.

Exp No:20 Experiment to understand K-means clustering algorithm for a given data set.

AIM:

To implement the k-means clustering algorithm on a given dataset.

Description:

Understand the K-means clustering algorithm for the dataset given.

CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
df=pd.read_csv('Mall_Customers.csv')
df.info()
df.head()
sns.pairplot(df)
features=df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
```

```
plt.show()

features_el=df.iloc[:,[2,3,4]].values

from sklearn.cluster import KMeans

wcss=[]

for i in range(1,10):

model=KMeans(n_clusters=i)

model.fit(features_el)

wcss.append(model.inertia_)

plt.plot(range(1,10),wcss)
```

OUTPUT:

1. Dataset Information

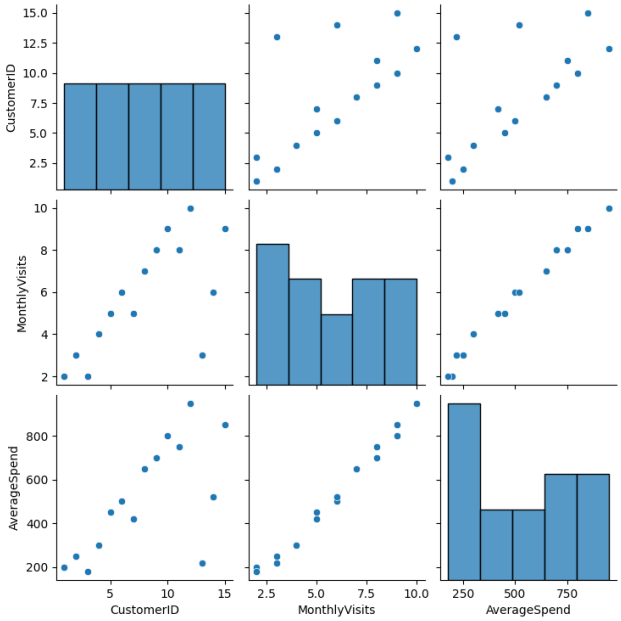
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 3 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   CustomerID    15 non-null    int64
 1   MonthlyVisits 15 non-null    int64
 2   AverageSpend  15 non-null    int64
dtypes: int64(3)
memory usage: 492.0 bytes
```

2. Original Dataset – First Five Records

	CustomerID	MonthlyVisits	Average Spend
0	1	2	200
1	2	3	250
2	3	2	180
3	4	4	300

4	5	5	450

3. Pair Plot



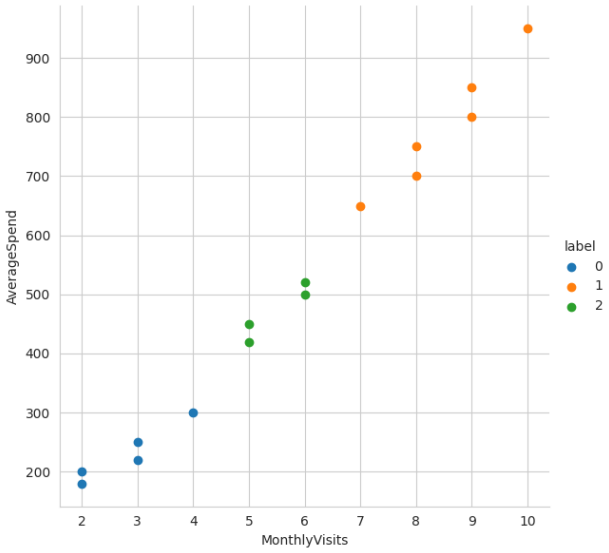
4. K-Means Model

KMeans(n_clusters=3, n_init=10, random_state=42)

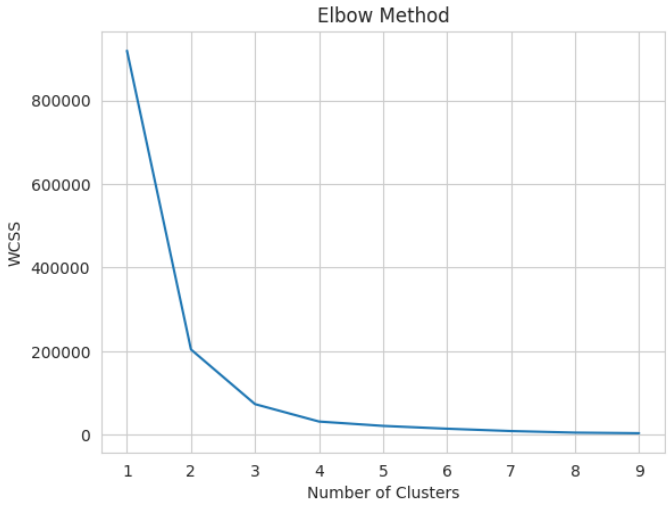
5. Clustered Data – First Five Records

	MonthlyVisits	AverageSpend	label
0	2	200	0
1	3	250	0
2	2	180	0
3	4	300	0
4	5	450	2

6. Cluster Visualization



7. Elbow Method – WCSS



8. Final K-Means Model

KMeans(n_clusters=3, n_init=10, random_state=42)

9. Final Clustered Dataset

	MonthlyVisits	AverageSpend	label
0	2	200	0
1	3	250	0
2	2	180	0
3	4	300	0
4	5	450	2
5	6	500	2
6	5	420	2

7	7	650	1
8	8	700	1
9	9	800	1
10	8	750	1
11	10	950	1
12	3	220	0
13	6	520	2
14	9	850	1

10. Final Cluster Output

Customer groups are represented by the labels 0, 1 and 2.
The final dataset shows the cluster assigned to each customer based on MonthlyVisits and AverageSpend.

RESULT:

The café customers were successfully grouped into 3 clusters using k-means clustering.